Universidad Tecnologica
de Xicoptec de Juarez

UTXJ
TECNOLOGÍAS DE LA INFORMACIÓN

Guía de Instalación de Entorno para Desarrollo Móvil Multiplataforma



PRESENTADO POR:

- JOSE FRANCISCO FLORES AMADOR
- EDGAR CABRERA VELÁZQUEZ
- EDWIN HERNÁNDEZ CAMPOS
- GIOVANY RAUL PAZOS CRUZ
- BRISA NALLELY GARCIA GREGORIO

Desarollo Movil Integral

Tabla de Contenido

Tabla de Contenido 2
Introducción 4
Objetivo General 4
Objetivos Específicos 4
Capítulo 1. Marco Teórico 5
1.1 Arquitectura de aplicaciones móviles multiplataforma 5
1.2 Flutter y Dart 5
1.3 Autenticación y gestión de identidad con Keycloak 5
1.4 Documentación de APIs con Swagger/OpenAPI 5
1.5 Frameworks de backend: FastAPI y Node/Express 5
1.6 Bases de datos relacionales y no relacionales 5
1.7 Contenerización con Docker y Docker Compose 5
1.8 Control de versiones con Git y GitHub 6
1.9 Pruebas de API con Postman 6
Capítulo 2. Requisitos Previos 7
2.1 Requisitos de hardware y sistema operativo 7
2.2 Cuentas y accesos necesarios 7
2.3 Tabla de versiones recomendadas 7
Capítulo 3. Instalación de Herramientas Base 8
3.1 Instalación de Git 8
Windows / macOS / Linux 8
Configuración de llave SSH y conexión con GitHub 9
3.2 Instalación de Node.js y npm 9
3.3 Instalación de Docker y Docker Compose 10
3.4 Instalación del Flutter SDK 13
3.5 Instalación del IDE y plugins 18
3.6 Instalación de Postman 19
Capítulo 4. Configuración del Entorno Backend 24
4.1 Configuración de PostgreSQL 24
4.2 Configuración de MongoDB 25
4.3 Configuración de Keycloak 26
Creación de Realm, Cliente y Usuario de prueba 28
4.4 Configuración de FastAPI 29
4.5 Configuración alterna con Node/Express 30
4.6 Integración de Swagger/OpenAPI 31
4.7 Orquestación completa con Docker Compose 31
Capítulo 5. Configuración del Entorno Frontend (Flutter) 32

5.1 Creación del proyecto Flutter 32
5.2 Estructura de carpetas recomendada 33
5.3 Integración de paquetes para consumo de API 33
5.4 Integración de autenticación con Keycloak 34
5.5 Configuración de variables de entorno por ambiente 35
Capítulo 6. Pruebas y Verificación 36
6.1 Pruebas de endpoints con Postman 36
6.2 Pruebas de autenticación (obtención y validación de JWT) 37
6.3 Verificación de la documentación Swagger 38
6.4 Pruebas de la app Flutter contra el backend 39
Capítulo 7. Buenas Prácticas 40
7.1 Manejo de variables sensibles 40
7.2 Convenciones de commits y ramas 40
7.3 Versionado semántico de la API 40
Capítulo 8. Solución de Problemas Comunes 41
Anexos 42
Anexo A. Comandos rápidos de referencia 42
Anexo B. Ejemplo completo de docker-compose.yml 42
Anexo C. Glosario de términos 42
Anexo D. Referencias 43

Introducción

El presente manual describe, de forma teórica y técnica, el proceso de instalación y configuración de un entorno de desarrollo completo para la construcción de aplicaciones móviles multiplataforma. El entorno integra un frontend desarrollado en Flutter, un backend basado en FastAPI (con una alternativa en Node/Express), autenticación centralizada mediante Keycloak, dos motores de base de datos (PostgreSQL y MongoDB), documentación de API con Swagger/OpenAPI, pruebas de endpoints con Postman, contenerización con Docker y Docker Compose, y control de versiones con Git y GitHub.

Este documento está pensado como una guía de referencia paso a paso: cada sección combina la explicación conceptual de la herramienta (qué es y para qué sirve) con el procedimiento técnico de instalación y configuración, dejando espacios reservados para insertar capturas de pantalla propias del proceso realizado.

Objetivo General

Instalar y configurar correctamente todas las herramientas y tecnologías necesarias para el desarrollo de una aplicación móvil multiplataforma con arquitectura cliente-servidor, autenticación centralizada y despliegue contenerizado.

Objetivos Específicos

- Comprender el rol teórico de cada herramienta dentro de la arquitectura general del sistema.
- Instalar y verificar el correcto funcionamiento de Git, Node.js, Docker y Flutter SDK.
- Configurar contenedores de PostgreSQL, MongoDB y Keycloak mediante Docker Compose.
- Levantar un backend con FastAPI (o Node/Express) documentado con Swagger/OpenAPI.
- Validar la comunicación entre el backend y las bases de datos mediante Postman.
- Conectar la aplicación Flutter con el backend y el proveedor de identidad Keycloak.

Capítulo 1. Marco Teórico

1.1 Arquitectura de aplicaciones móviles multiplataforma

El desarrollo multiplataforma permite crear una única base de código que se ejecuta en distintos sistemas operativos (Android, iOS, web, escritorio). A diferencia del desarrollo nativo, que exige mantener proyectos separados por plataforma, frameworks como Flutter compilan a código nativo desde un mismo lenguaje (Dart), reduciendo tiempo y costo de desarrollo.

La arquitectura típica sigue el patrón cliente-servidor: el cliente (app móvil) consume servicios expuestos por un servidor (API REST), el cual gestiona la lógica de negocio, la persistencia de datos y la seguridad.

1.2 Flutter y Dart

Flutter es un SDK de código abierto creado por Google para construir interfaces nativas compiladas para móvil, web y escritorio a partir de un único código base escrito en Dart. Su unidad fundamental es el widget: todo elemento visual (texto, botón, layout) es un widget, organizado en un árbol jerárquico. Una de sus características distintivas es el hot reload, que permite ver cambios de código reflejados en la app en ejecución sin perder el estado.

1.3 Autenticación y gestión de identidad con Keycloak

Keycloak es un servidor de identidad y gestión de accesos de código abierto que implementa los estándares OAuth 2.0 y OpenID Connect (OIDC). Permite centralizar la autenticación (Single Sign-On), la autorización basada en roles y la emisión de tokens JWT (JSON Web Token) que las aplicaciones cliente y las APIs utilizan para validar la identidad del usuario sin necesidad de manejar contraseñas directamente.

1.4 Documentación de APIs con Swagger/OpenAPI

OpenAPI es una especificación estándar para describir APIs REST de forma legible tanto por humanos como por máquinas. Swagger es el conjunto de herramientas que implementa dicha especificación, generando documentación interactiva (Swagger UI) donde se pueden visualizar y probar los endpoints disponibles, sus parámetros y respuestas.

1.5 Frameworks de backend: FastAPI y Node/Express

FastAPI es un framework de Python de alto rendimiento para construir APIs, basado en tipado estático (type hints) y generación automática de documentación OpenAPI. Node/Express, por su parte, es un framework minimalista para JavaScript/Node.js ampliamente usado para construir APIs REST mediante middlewares y enrutadores. Ambos siguen el patrón arquitectónico REST y pueden coexistir como microservicios independientes.

1.6 Bases de datos relacionales y no relacionales

PostgreSQL es un sistema gestor de bases de datos relacional (SQL), que organiza la información en tablas con esquemas fijos y relaciones definidas mediante llaves foráneas; es ideal para datos estructurados que requieren integridad referencial. MongoDB es una base de datos no relacional (NoSQL) orientada a documentos, que almacena la información en formato BSON/JSON sin un esquema rígido, siendo adecuada para datos semiestructurados o con alta variabilidad.

1.7 Contenerización con Docker y Docker Compose

Docker es una plataforma de contenerización que empaqueta una aplicación junto con todas sus dependencias en una unidad estandarizada llamada contenedor, garantizando que se ejecute igual en cualquier entorno. A diferencia de una máquina virtual, un contenedor comparte el kernel del sistema operativo anfitrión, resultando más ligero y

rápido de iniciar. Docker Compose permite definir y orquestar múltiples contenedores (por ejemplo, backend, base de datos y proveedor de identidad) mediante un único archivo declarativo (docker-compose.yml).

1.8 Control de versiones con Git y GitHub

Git es un sistema de control de versiones distribuido que permite rastrear los cambios realizados sobre el código fuente a lo largo del tiempo, facilitando el trabajo colaborativo mediante ramas (branches) y fusiones (merges). GitHub es una plataforma en la nube que aloja repositorios Git y añade funcionalidades de colaboración como pull requests, issues y flujos de integración continua.

1.9 Pruebas de API con Postman

Postman es una herramienta que permite construir, probar y documentar peticiones HTTP hacia una API sin necesidad de escribir código cliente. Organiza las peticiones en colecciones, admite variables de entorno para alternar entre configuraciones (desarrollo, pruebas, producción) y permite automatizar pruebas mediante scripts.

Capítulo 2. Requisitos Previos

2.1 Requisitos de hardware y sistema operativo

- Procesador de 64 bits con soporte de virtualización habilitado en BIOS/UEFI.
- Mínimo 8 GB de RAM (se recomienda 16 GB para ejecutar varios contenedores simultáneamente).
- Al menos 15 GB de espacio libre en disco.
- Sistema operativo: Windows 10/11, macOS o una distribución Linux (Ubuntu recomendado).

2.2 Cuentas y accesos necesarios

- Cuenta de GitHub (para clonar repositorios y control de versiones).
- Cuenta de Docker Hub (opcional, para descargar/publicar imágenes).

2.3 Tabla de versiones recomendadas

Herramienta	Versión recomendada	Verificación
Git	2.4x o superior	git --version
Node.js	20.x LTS	node -v
Docker	26.x o superior	docker --version
Docker Compose	v2 (plugin)	docker compose version
Flutter SDK	3.2x (stable)	flutter doctor
Python	3.11 o superior	python --version
PostgreSQL	16.x (imagen Docker)	psql --version
MongoDB	7.x (imagen Docker)	mongosh --version
Keycloak	25.x (imagen Docker)	Consola web /admin
Postman	Última versión estable	postman --version

Capítulo 3. Instalación de Herramientas Base

3.1 Instalación de Git

Git es la base del flujo de trabajo colaborativo; se instala antes que cualquier otra herramienta.

Windows / macOS / Linux

```
# Verificar instalación
git –version

# Configuración inicial de usuario
git config –global user.name “Tu Nombre”
git config –global user.email “tu_correo@ejemplo.com”
```

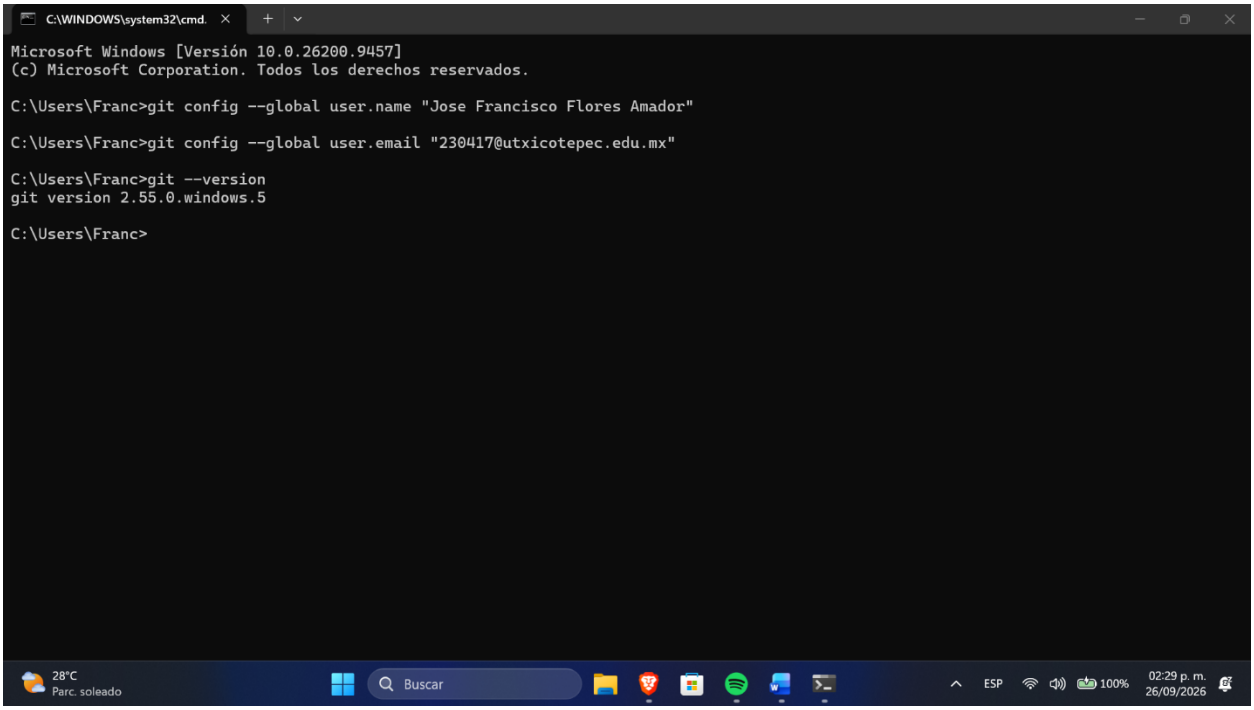


Figura 1. Instalación de Git y verificación de la versión con ‘git –version’

Configuración de llave SSH y conexión con GitHub

```
ssh-keygen -t ed25519 -C “tu_correo@ejemplo.com”
cat ~/.ssh/id_ed25519.pub
# Copiar la clave pública y agregarla en GitHub > Settings > SSH Keys
ssh -T git@github.com
```

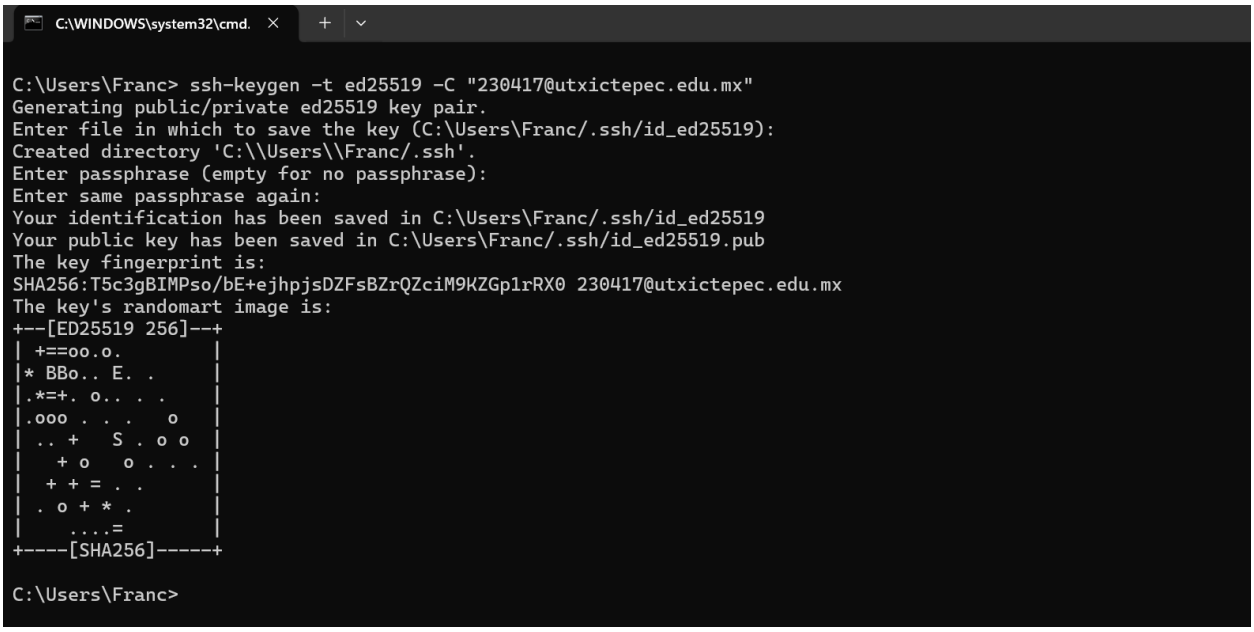


Figura 2. Clave SSH agregada en la configuración de GitHub

3.2 Instalación de Node.js y npm

Node.js es necesario para ejecutar herramientas de desarrollo, el backend en Express (si se usa) y utilidades de compilación.

```
node -v
npm -v
```

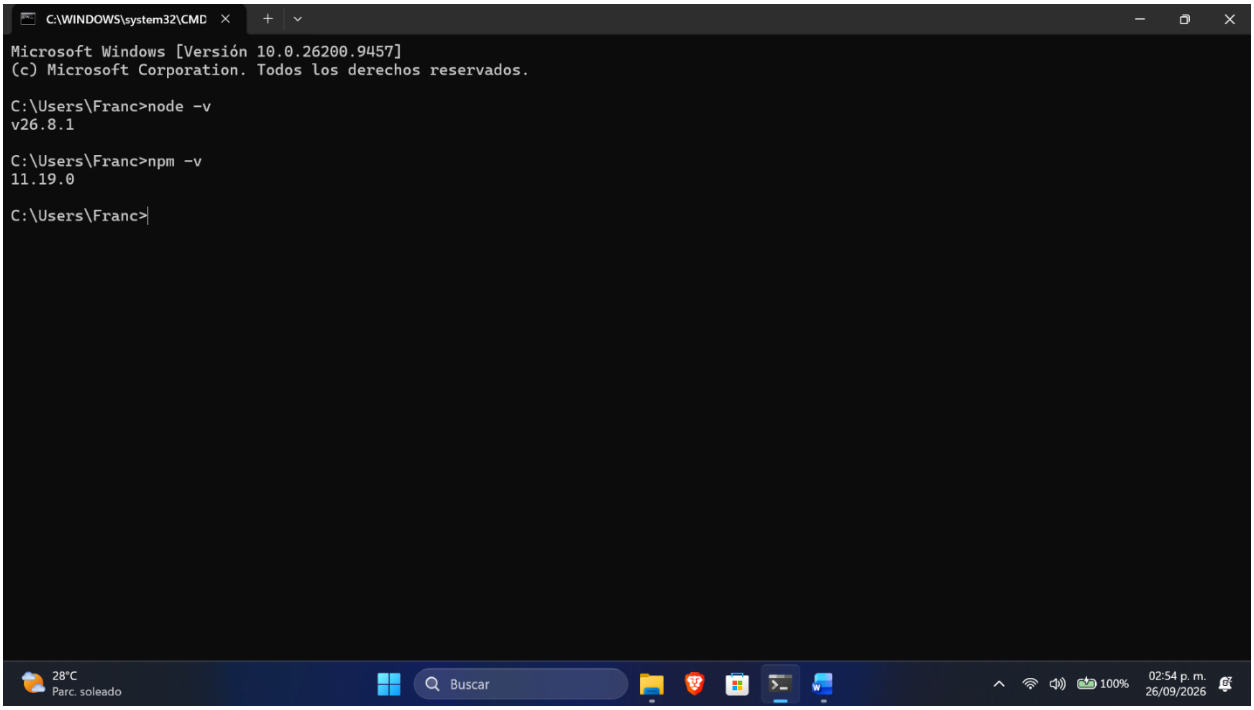
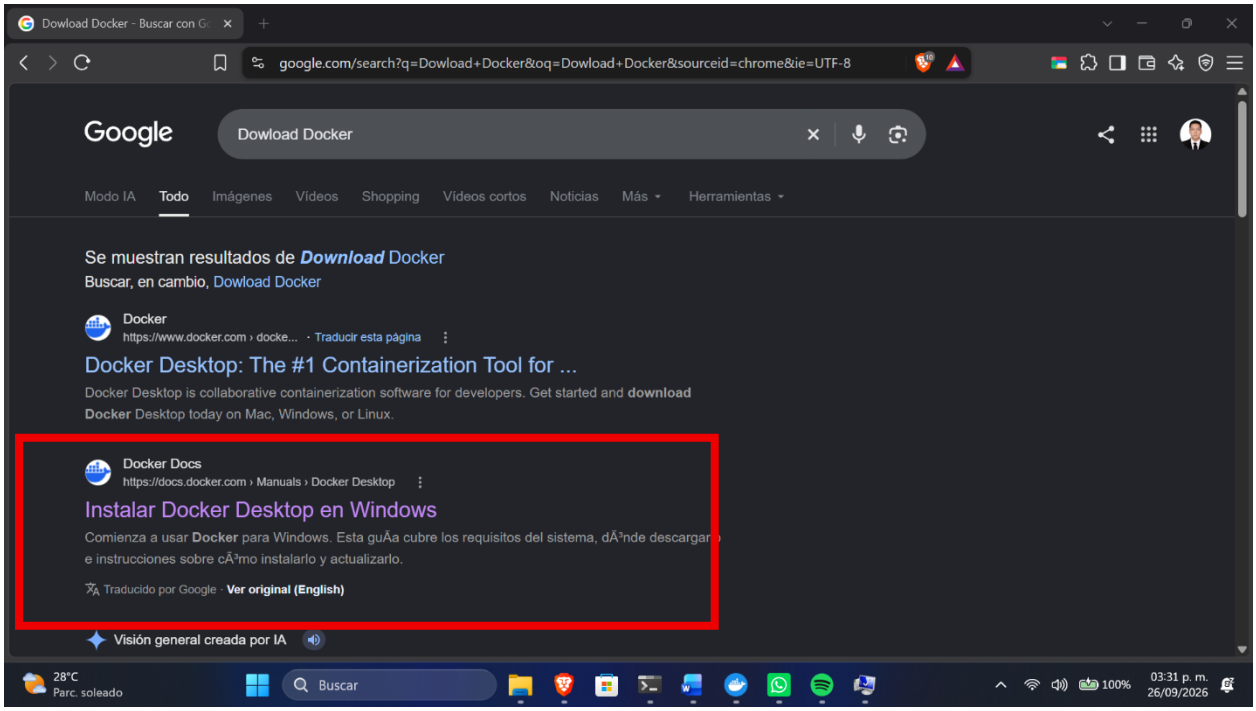


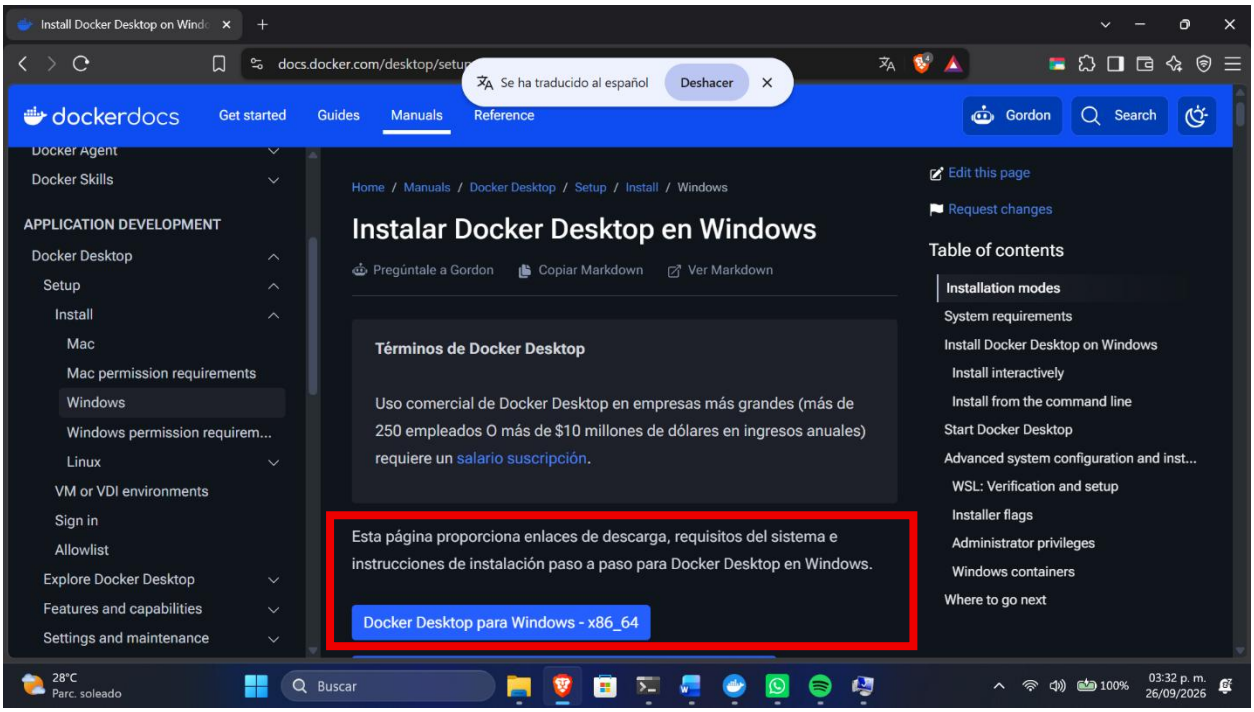
Figura 3. Verificación de versiones de Node.js y npm en terminal

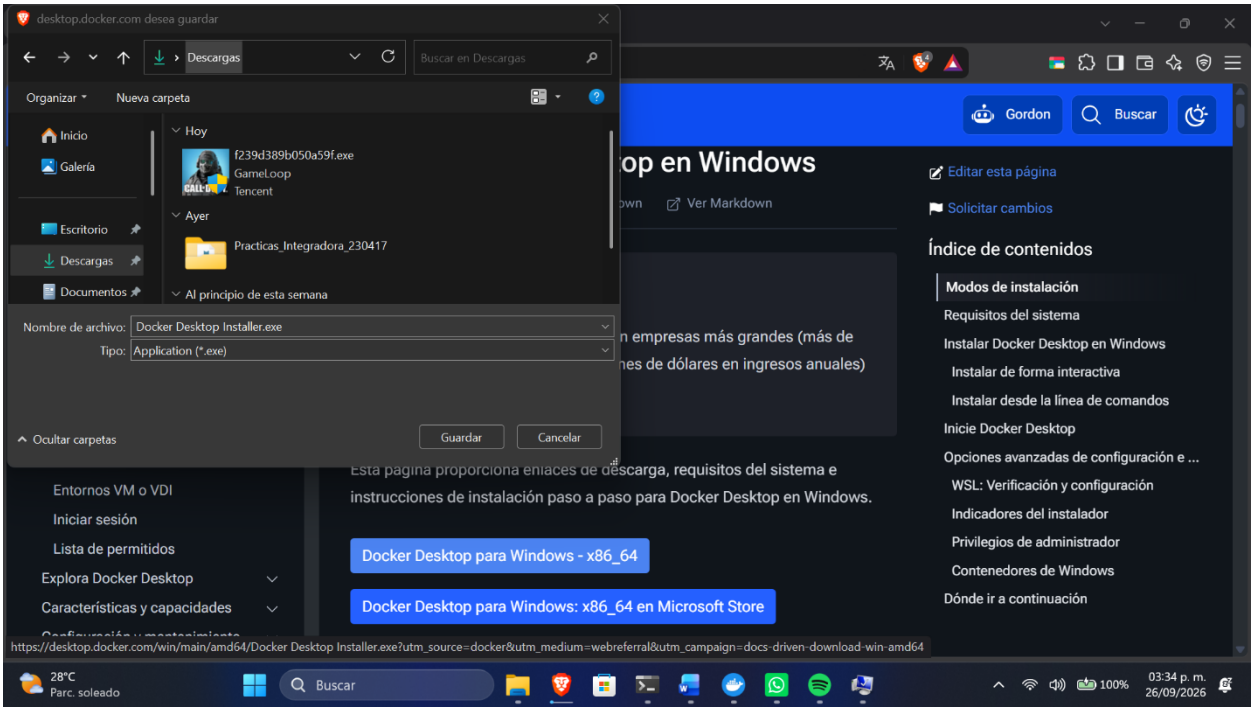
3.3 Instalación de Docker y Docker Compose

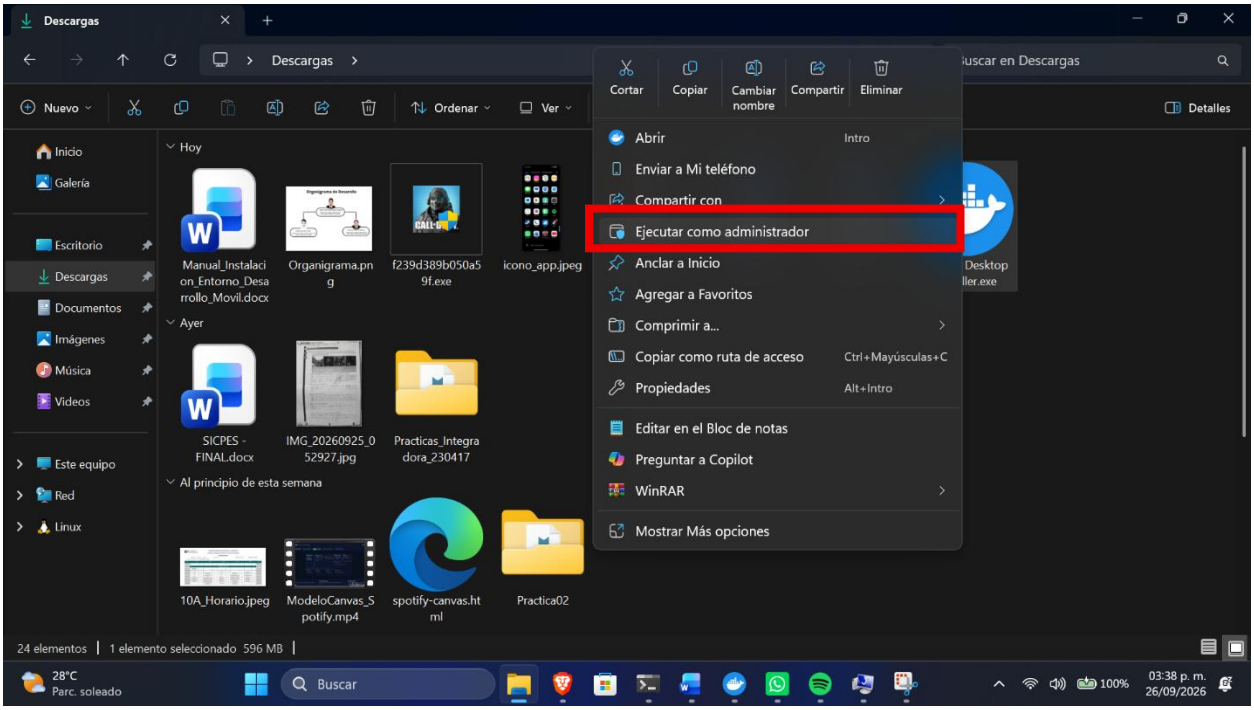
Docker Desktop (Windows/macOS) incluye Docker Engine y Docker Compose. En Linux se instalan por separado siguiendo la documentación oficial de Docker.



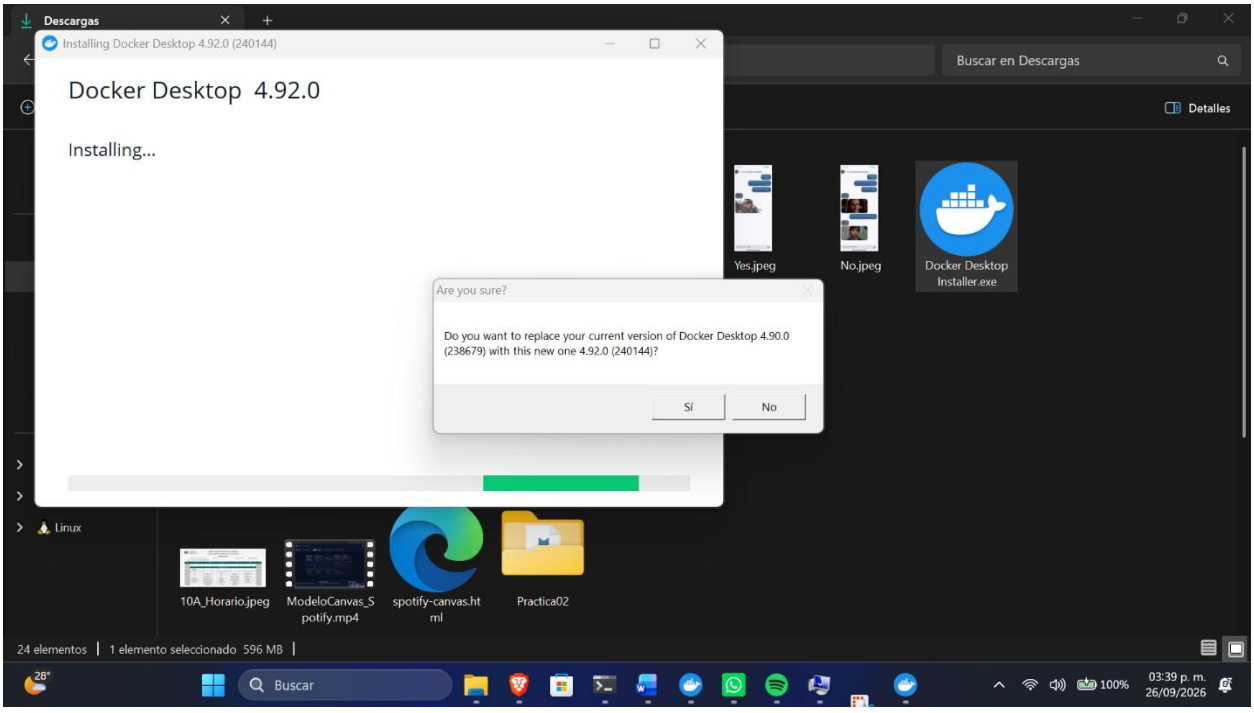
Descargamos Docker y lo guardamos en nuestras Descargas/Dowloads , la instalación es muy fácil y solo queda ejectar el programa como administrador y seguir los pasos de instalación los cuales son sencillos







En caso de ya tenerlo instalado, le saldrá está ventana para preguntar si desea actualizarlo o reinstalarlo



Verificación de Docker

```
docker --version
docker compose version
docker run hello-world
```

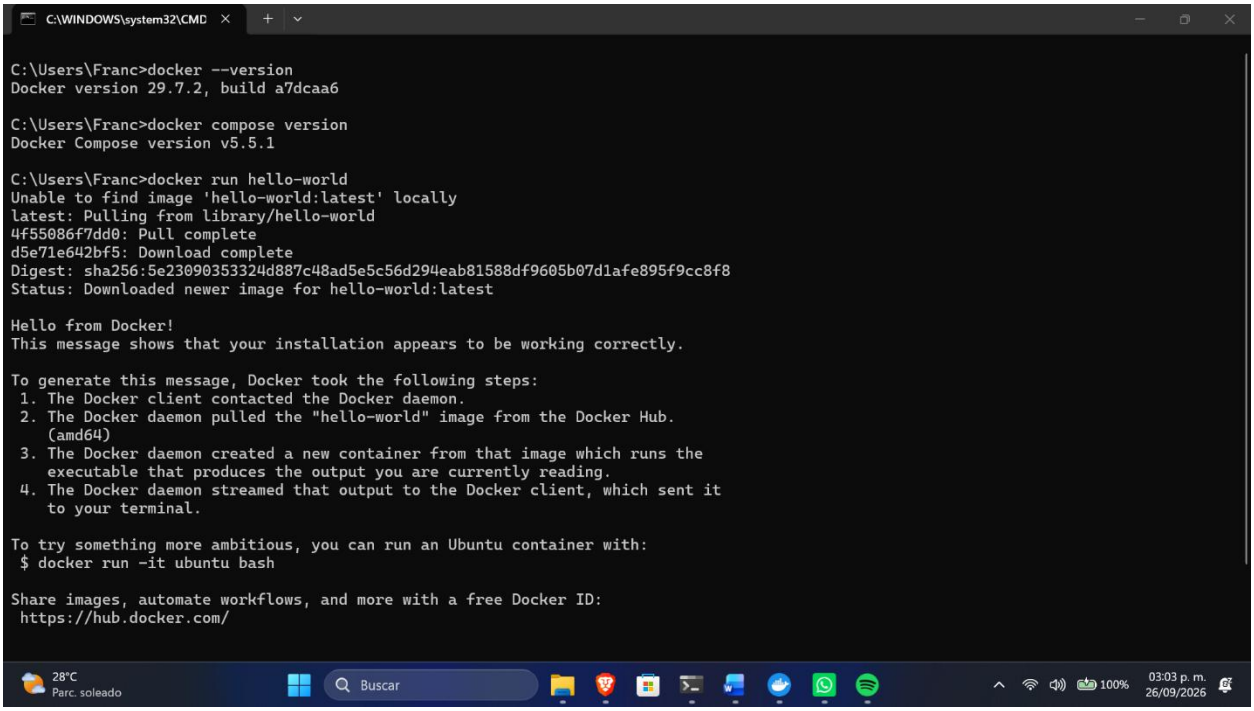
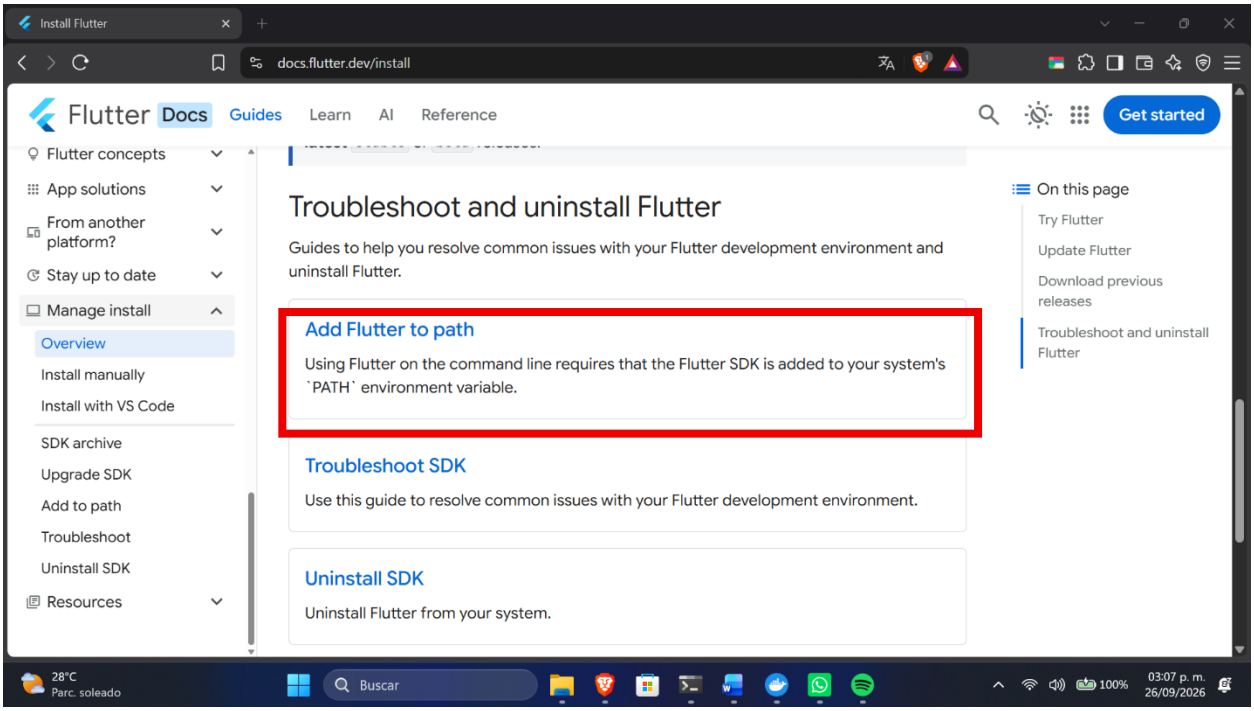


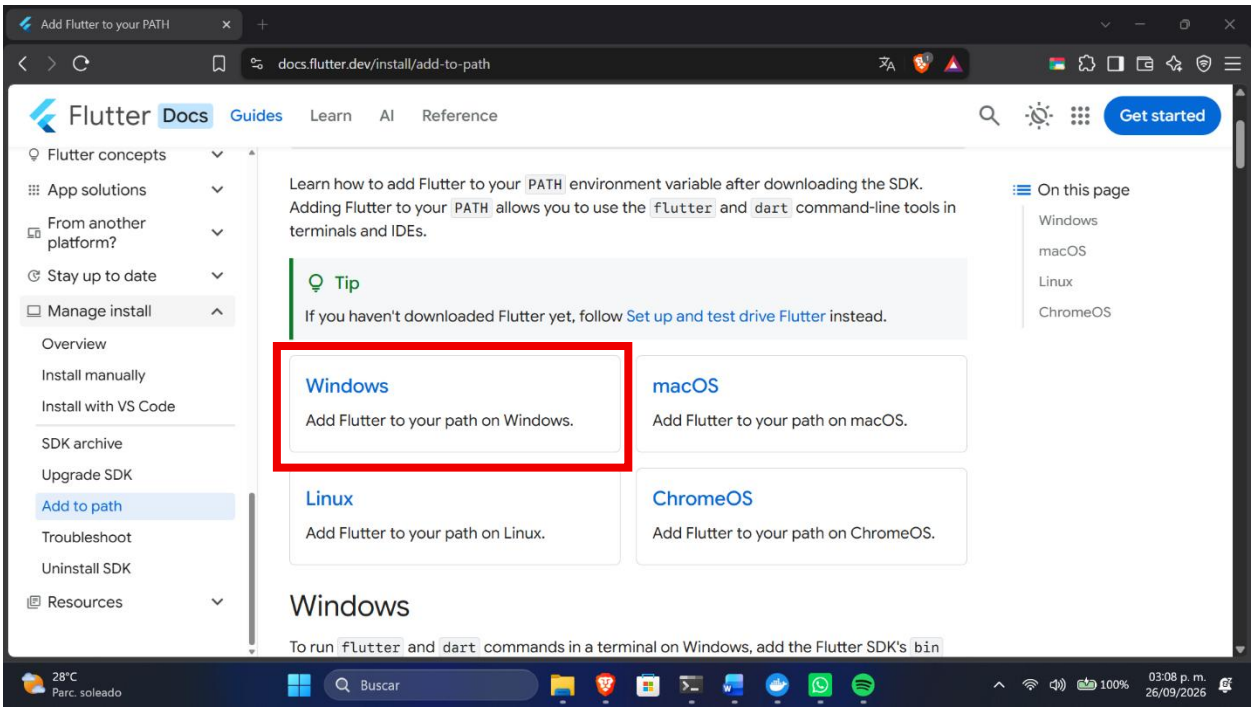
Figura 4. Salida del comando 'docker run hello-world' confirmando la instalación

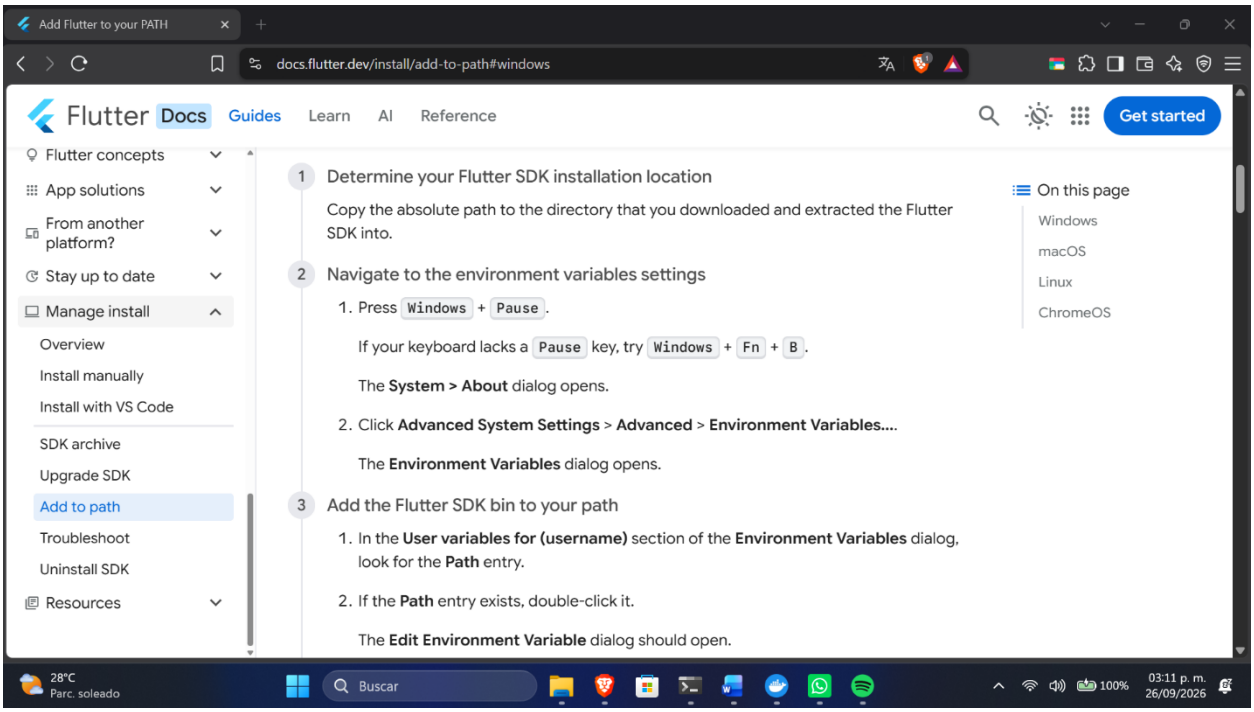
> **Nota:** Verifica que la virtualización (Hyper-V/WSL2 en Windows, o KVM en Linux) esté habilitada antes de instalar Docker Desktop.

3.4 Instalación del Flutter SDK

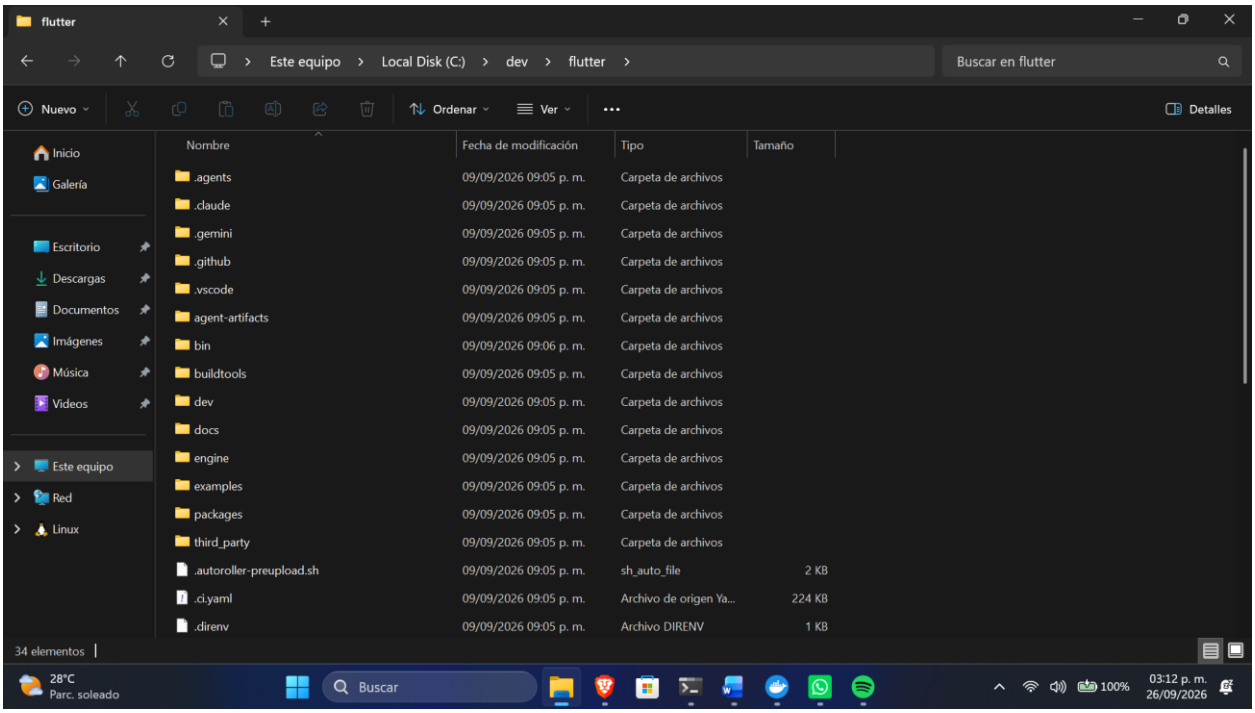
Se descarga el SDK desde el sitio oficial de Flutter, se agrega la carpeta bin al PATH del sistema y se ejecuta flutter doctor para validar dependencias (Android SDK, Xcode, licencias, etc.).


Install Flutter
docs.flutter.dev/install
Flutter Docs
Guides
Learn
AI
Reference
Get started
Flutter concepts
App solutions
From another platform?
Stay up to date
Manage install
Overview
Install manually
Install with VS Code
SDK archive
Upgrade SDK
Add to path
Troubleshoot
Uninstall SDK
Resources
Troubleshoot and uninstall Flutter
Guides to help you resolve common issues with your Flutter development environment and uninstall Flutter.
Add Flutter to path
Using Flutter on the command line requires that the Flutter SDK is added to your system's `PATH` environment variable.
Troubleshoot SDK
Use this guide to resolve common issues with your Flutter development environment.
Uninstall SDK
Uninstall Flutter from your system.
On this page
Try Flutter
Update Flutter
Download previous releases
Troubleshoot and uninstall Flutter
28°C
Parc. soleado
Buscar
100%
03:07 p. m.
26/09/2026


Add Flutter to your PATH
docs.flutter.dev/install/add-to-path
Flutter Docs
Guides
Learn
AI
Reference
Get started
Flutter concepts
App solutions
From another platform?
Stay up to date
Manage install
Overview
Install manually
Install with VS Code
SDK archive
Upgrade SDK
Add to path
Troubleshoot
Uninstall SDK
Resources
Learn how to add Flutter to your PATH environment variable after downloading the SDK. Adding Flutter to your PATH allows you to use the flutter and dart command-line tools in terminals and IDEs.
Tip
If you haven't downloaded Flutter yet, follow Set up and test drive Flutter instead.
Windows
Add Flutter to your path on Windows.
macOS
Add Flutter to your path on macOS.
Linux
Add Flutter to your path on Linux.
ChromeOS
Add Flutter to your path on ChromeOS.
Windows
To run flutter and dart commands in a terminal on Windows, add the Flutter SDK's bin
On this page
Windows
macOS
Linux
ChromeOS
28°C
Parc. soleado
Buscar
100%
03:08 p. m.
26/09/2026



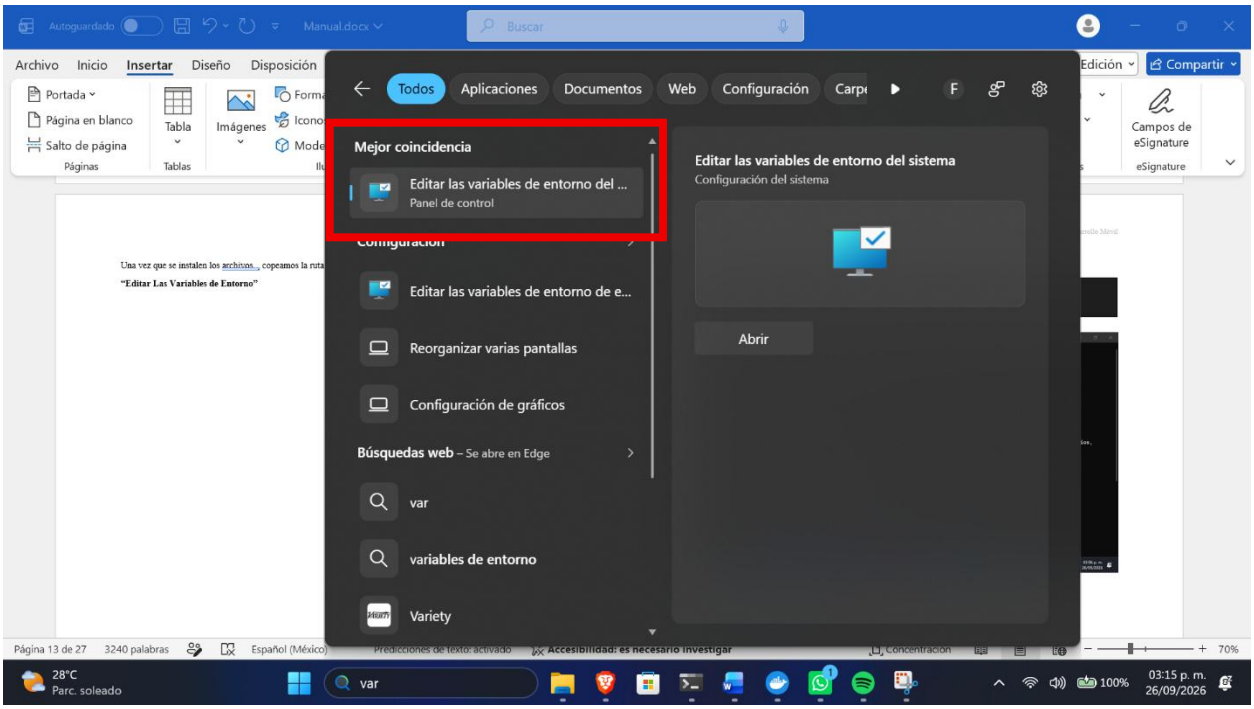
Se crea una carpeta con el nombre dev/flutter y se instala el SDK siguiendo los pazos de la página oficial de Flutter

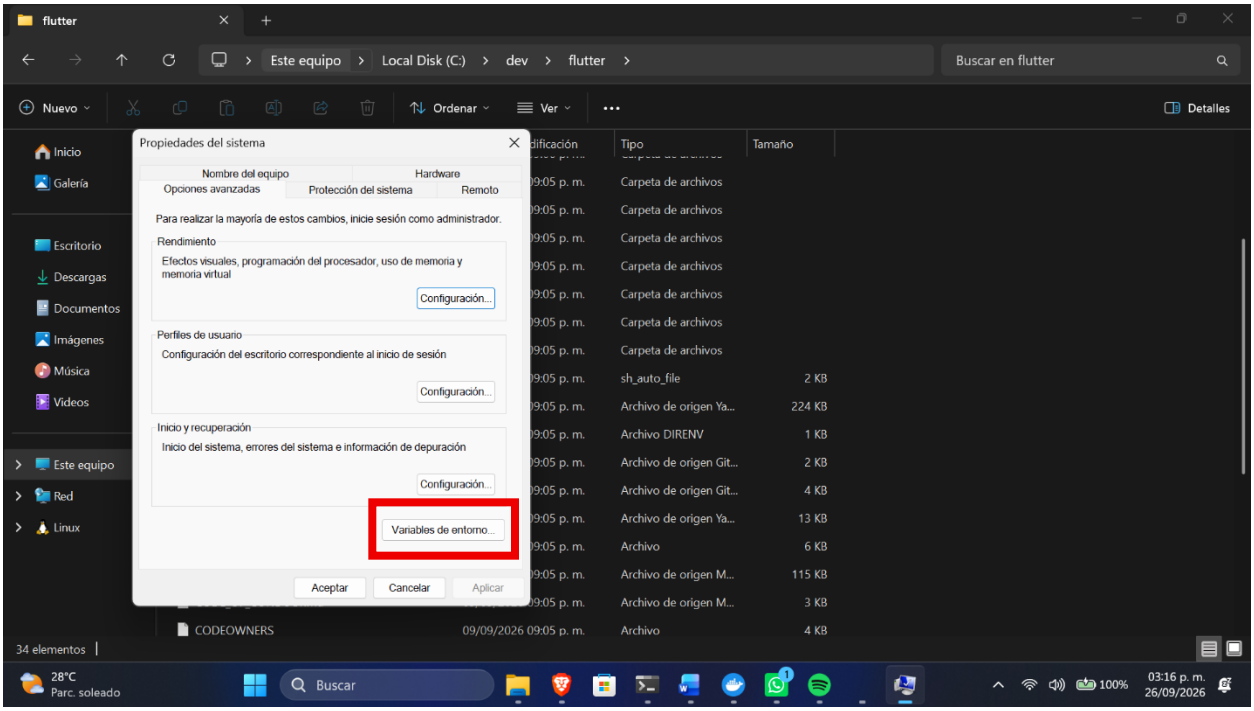


Ruta de instalación C:\dev\flutter

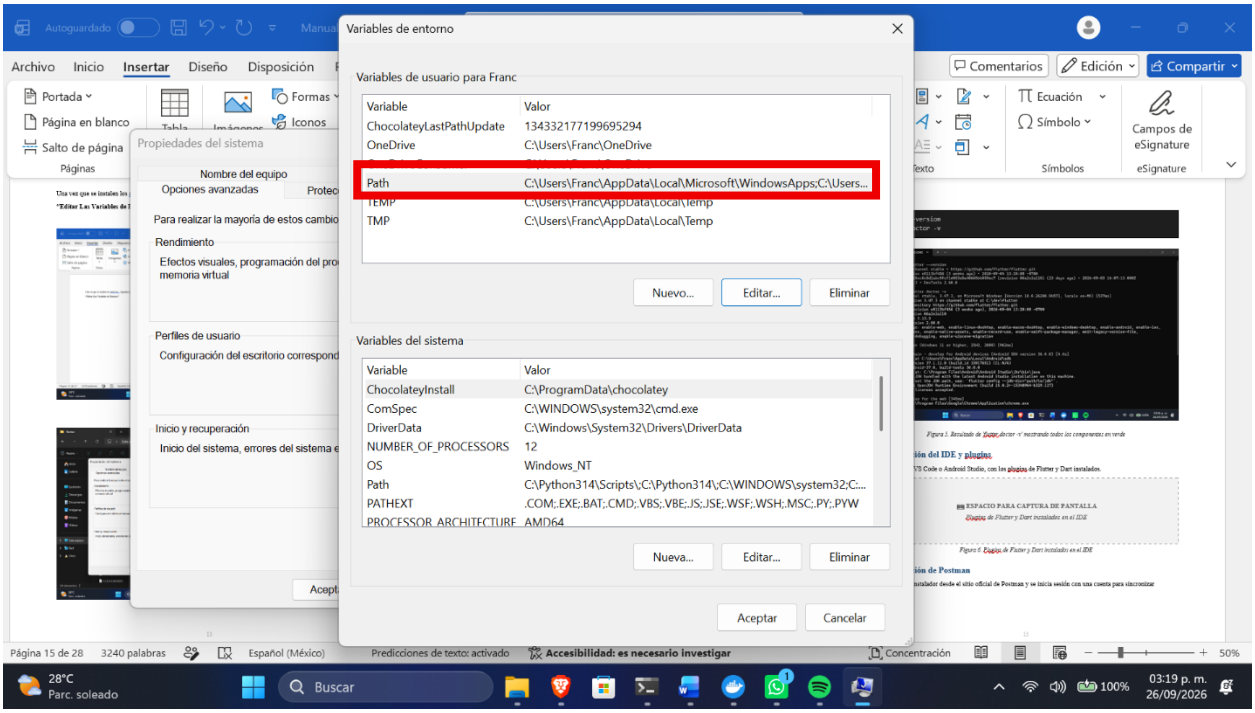
Una vez que se instalen los archivos , copeamos la ruta y en el buscador buscamos (valga la redundancia)

“Editar Las Variables de Entorno”

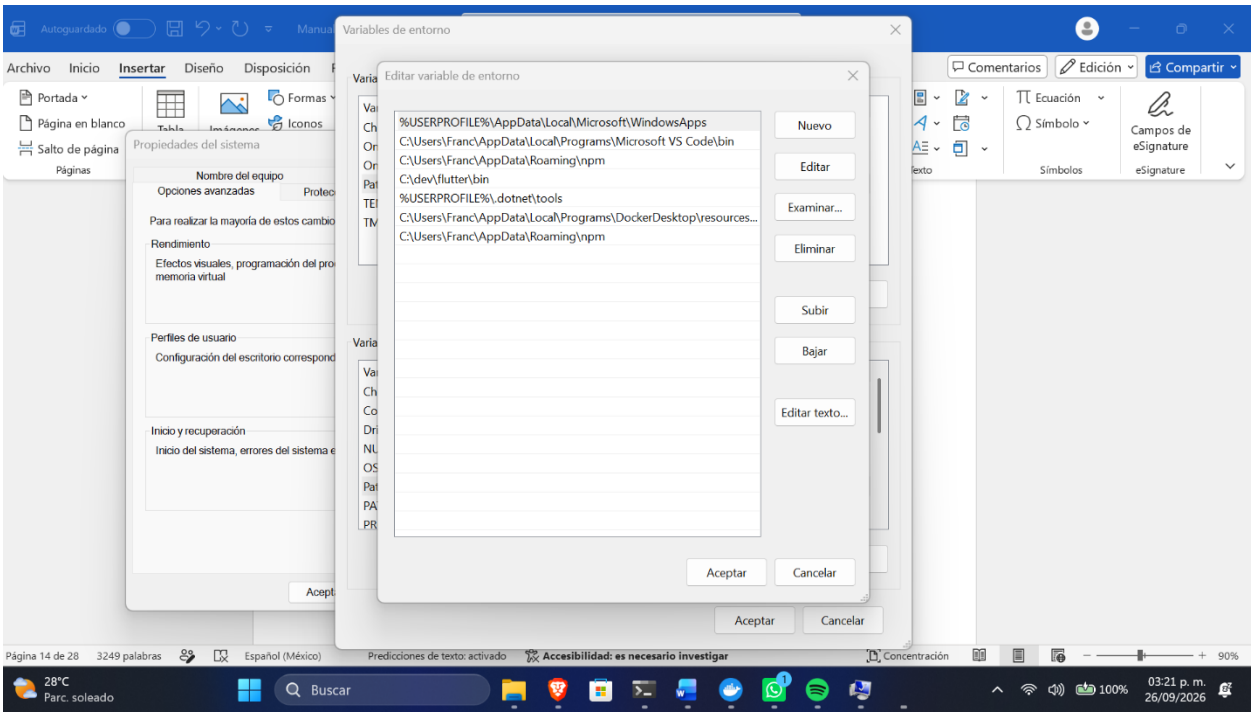




Buscamos la opción **Path** y damos clic en editar



Y pegamos la ruta que copeamos hace rato y guardamos los cambios



Verificacion de la versión de Flutter

```
flutter --version
flutter doctor -v
```

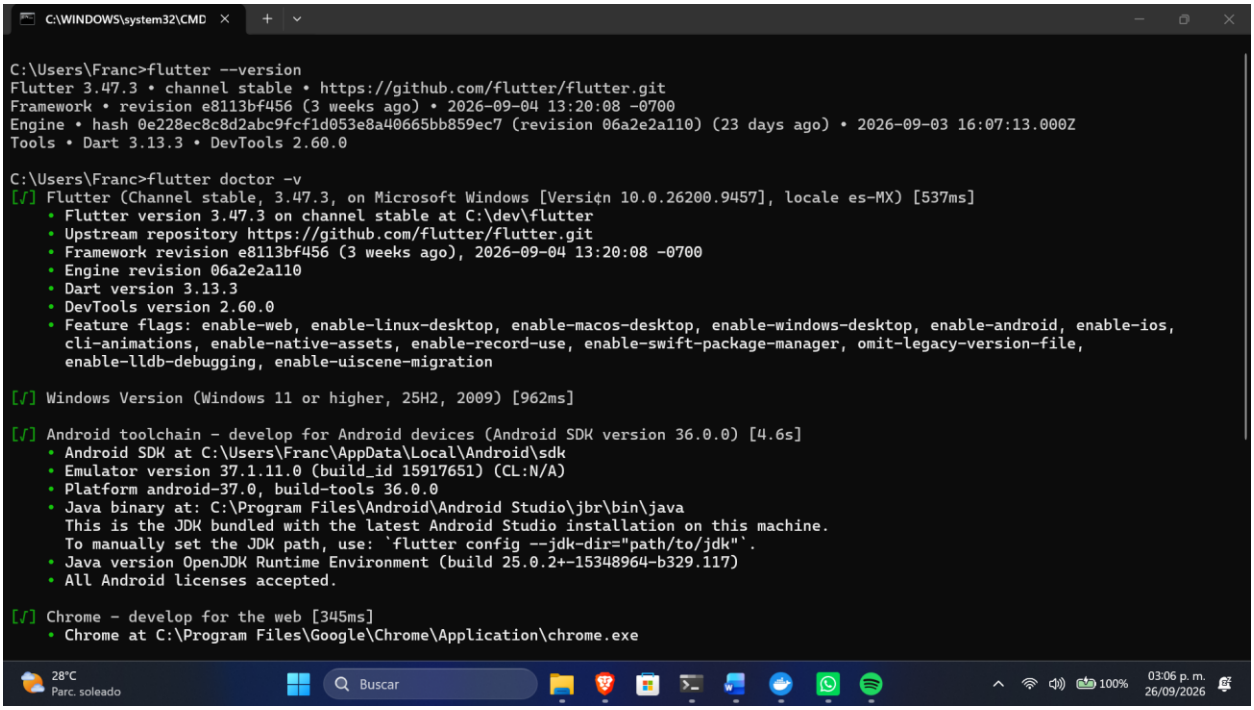
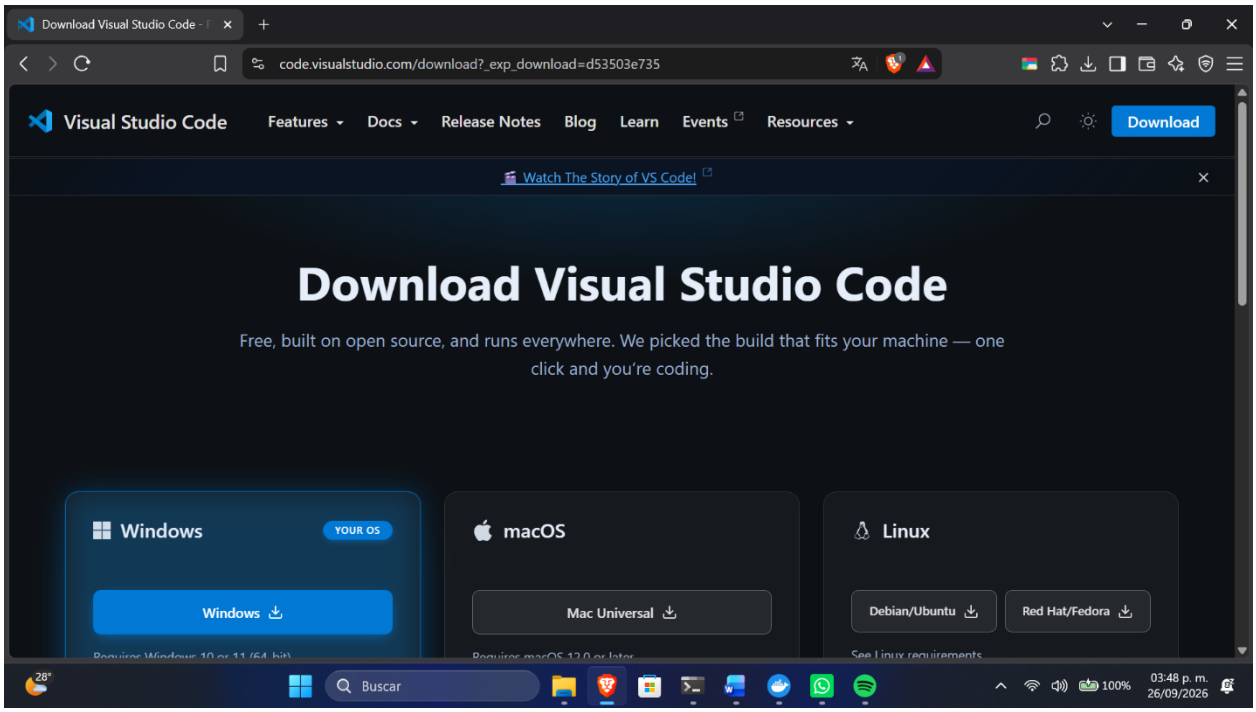


Figura 5. Resultado de 'flutter doctor -v' mostrando todos los componentes en verde

3.5 Instalación del IDE y plugins

Se recomienda VS Code o Android Studio, con los plugins de Flutter y Dart instalados.



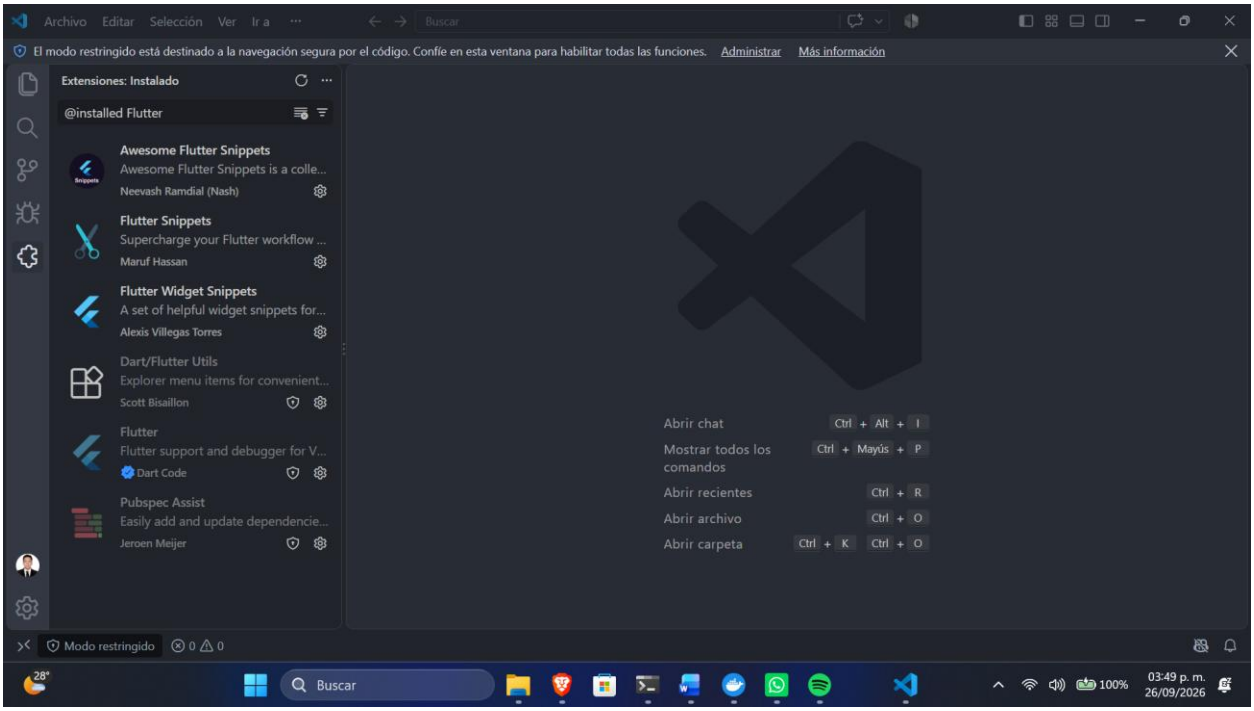
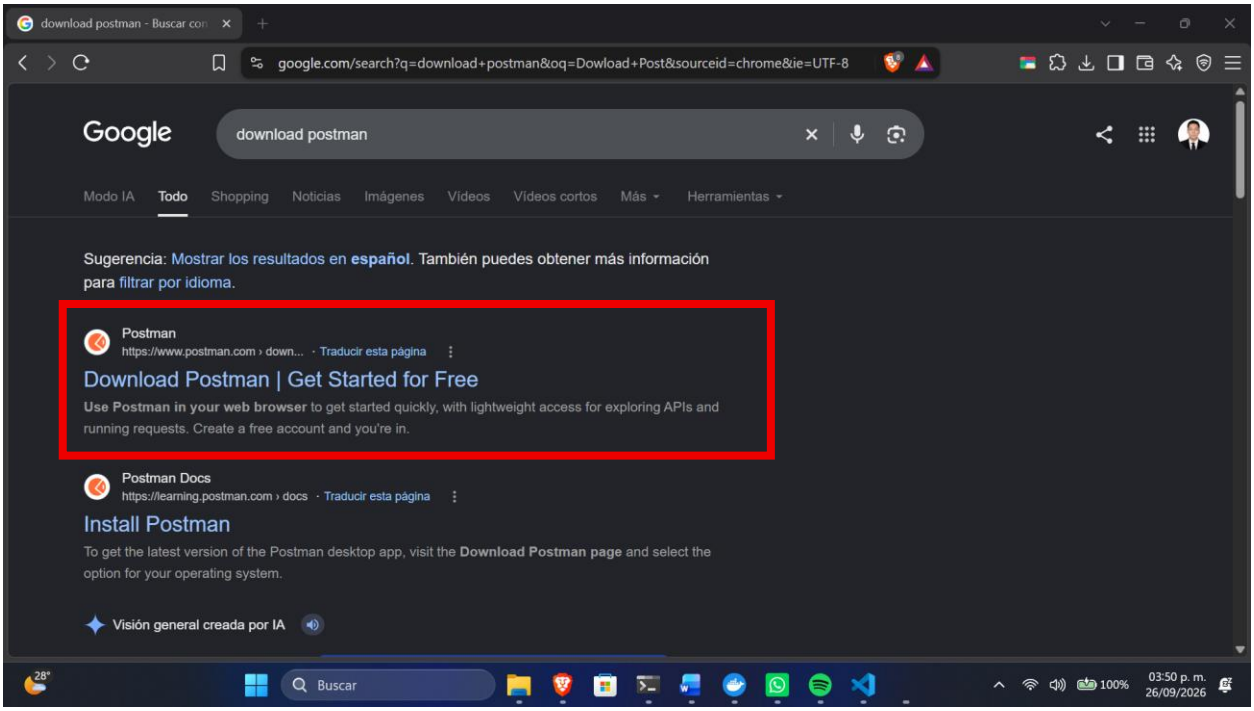


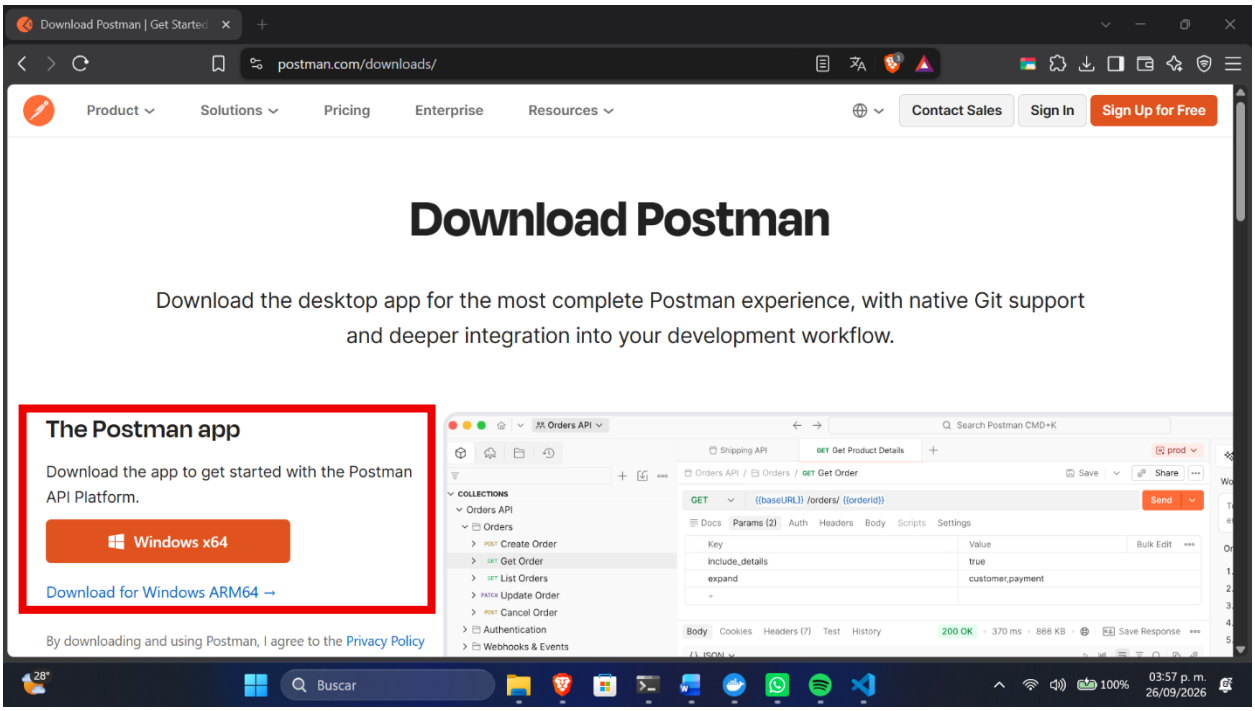
Figura 6. Plugins de Flutter y Dart instalados en el IDE

3.6 Instalación de Postman

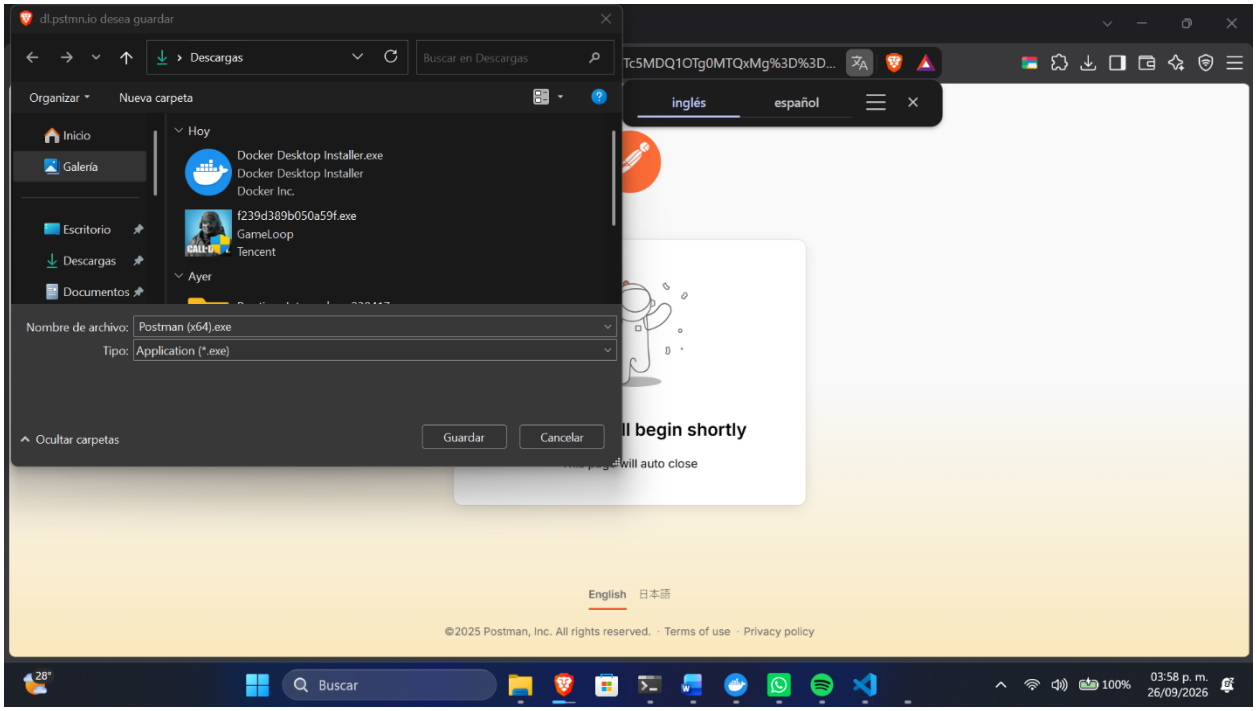
Se descarga el instalador desde el sitio oficial de Postman y se inicia sesión con una cuenta para sincronizar colecciones.



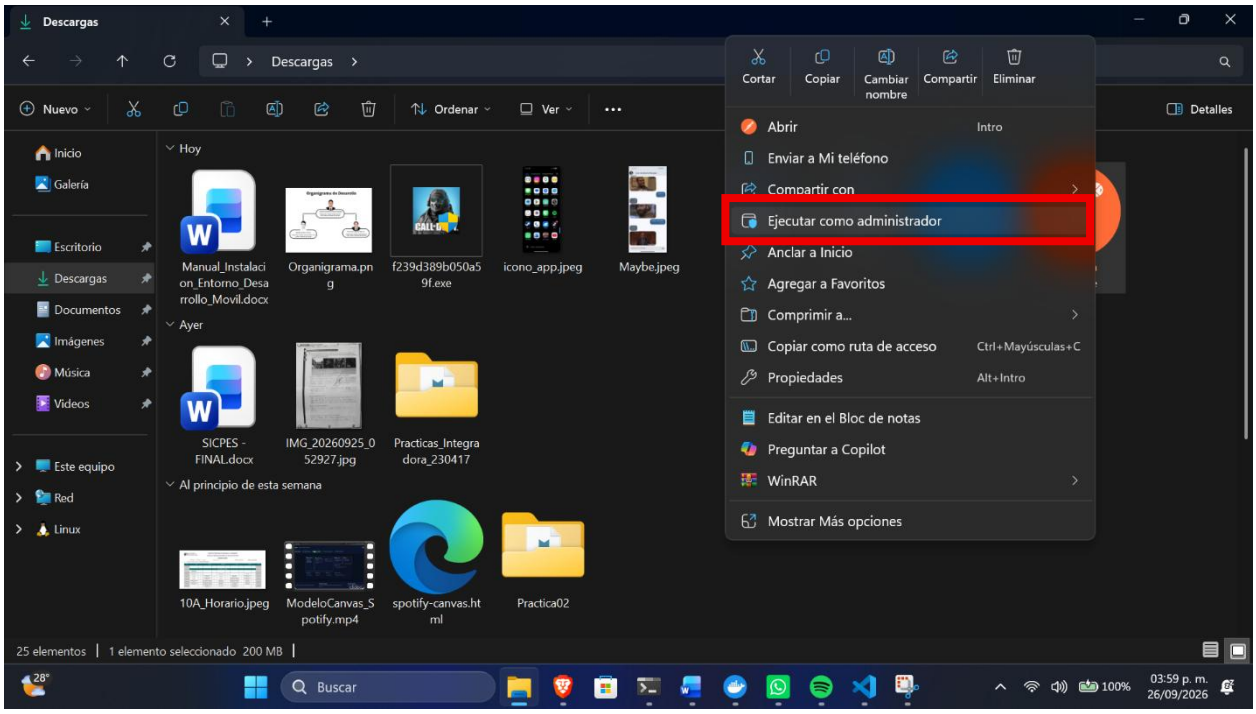
Damos clic en Windows x64 para que comience la descarga



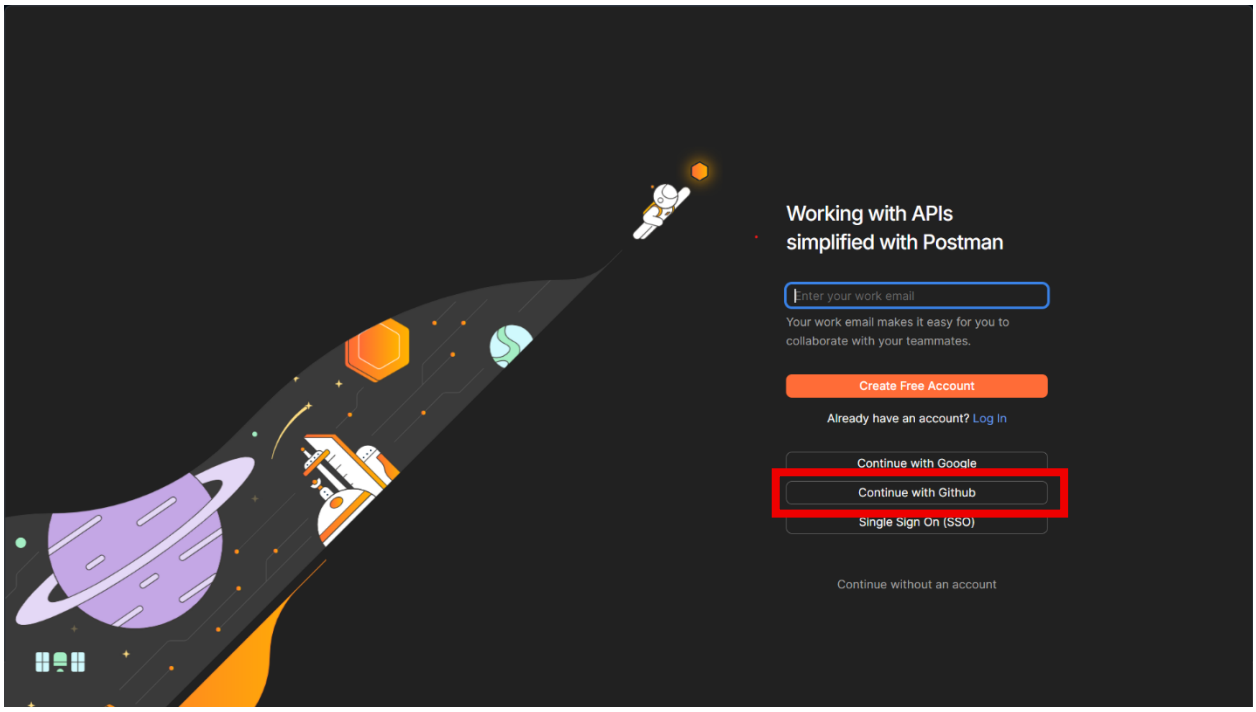
Lo guardamos en nuestro directorio de descargas

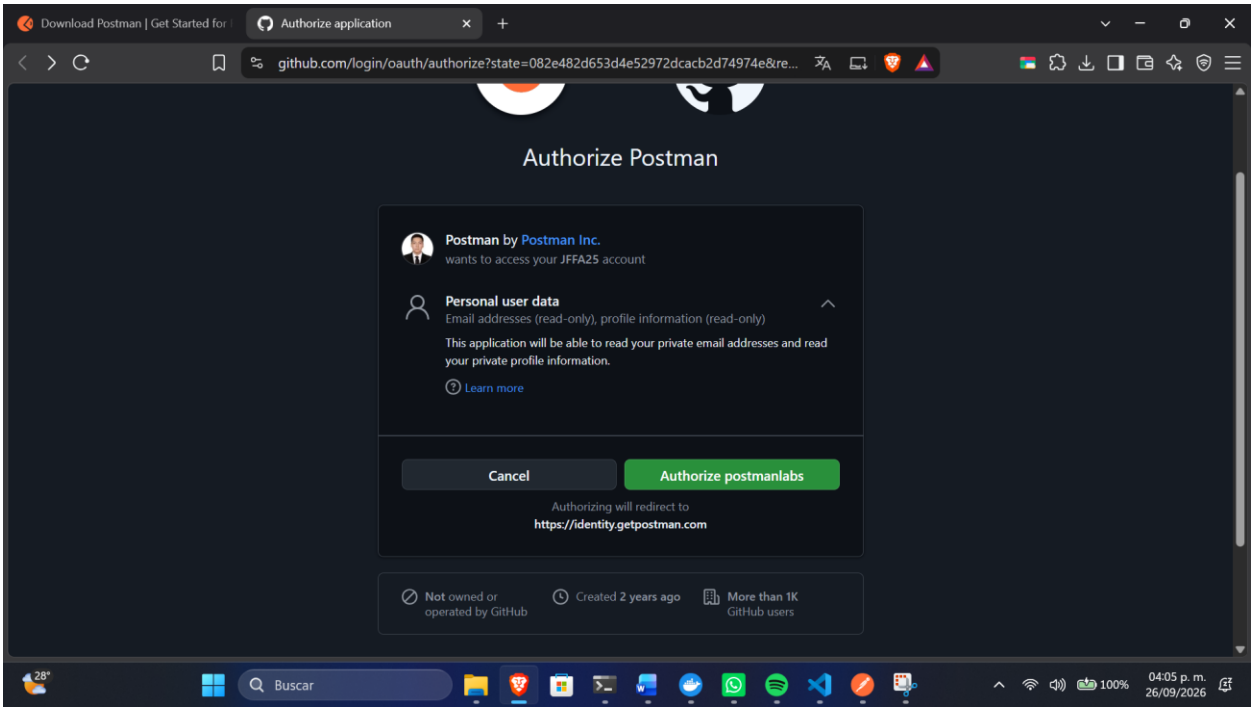


Ejecutamos como administrador

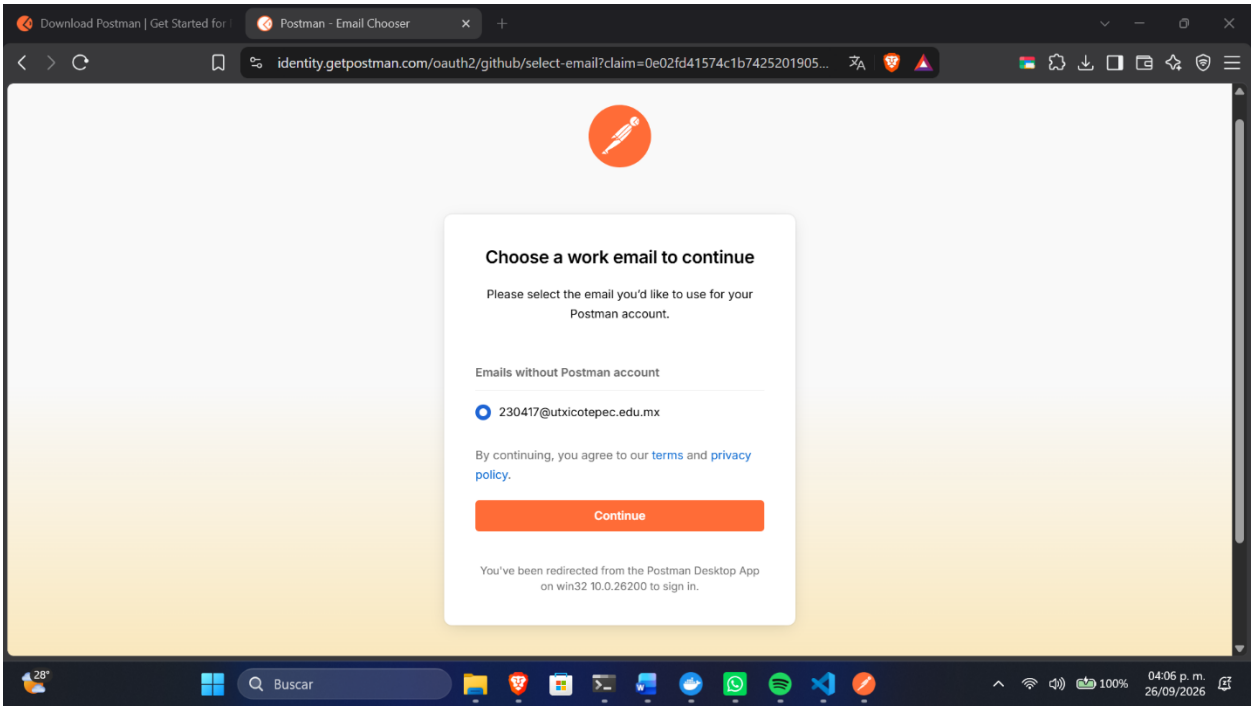


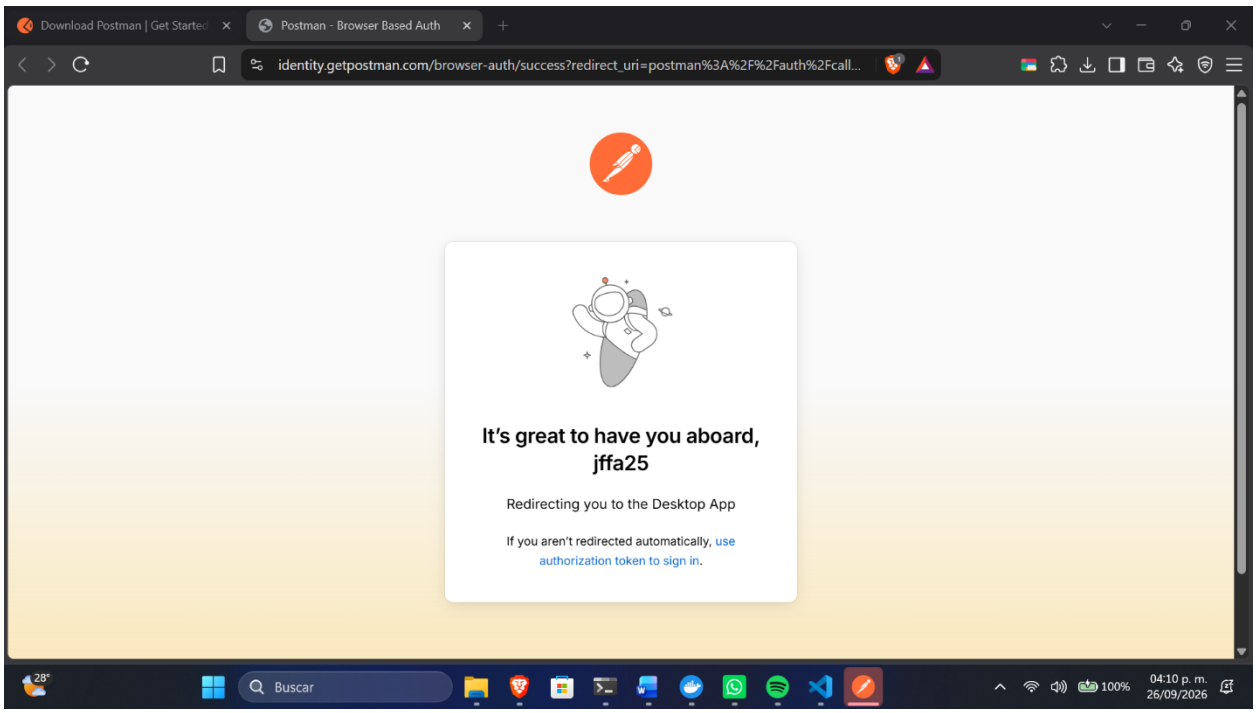
Seleccionamos continuar con GitHub (puede ser incluso con una cuenta de Google)





Una vez autorizado, damos clic en continuar





Y listo, ya tendremos POSTMAN

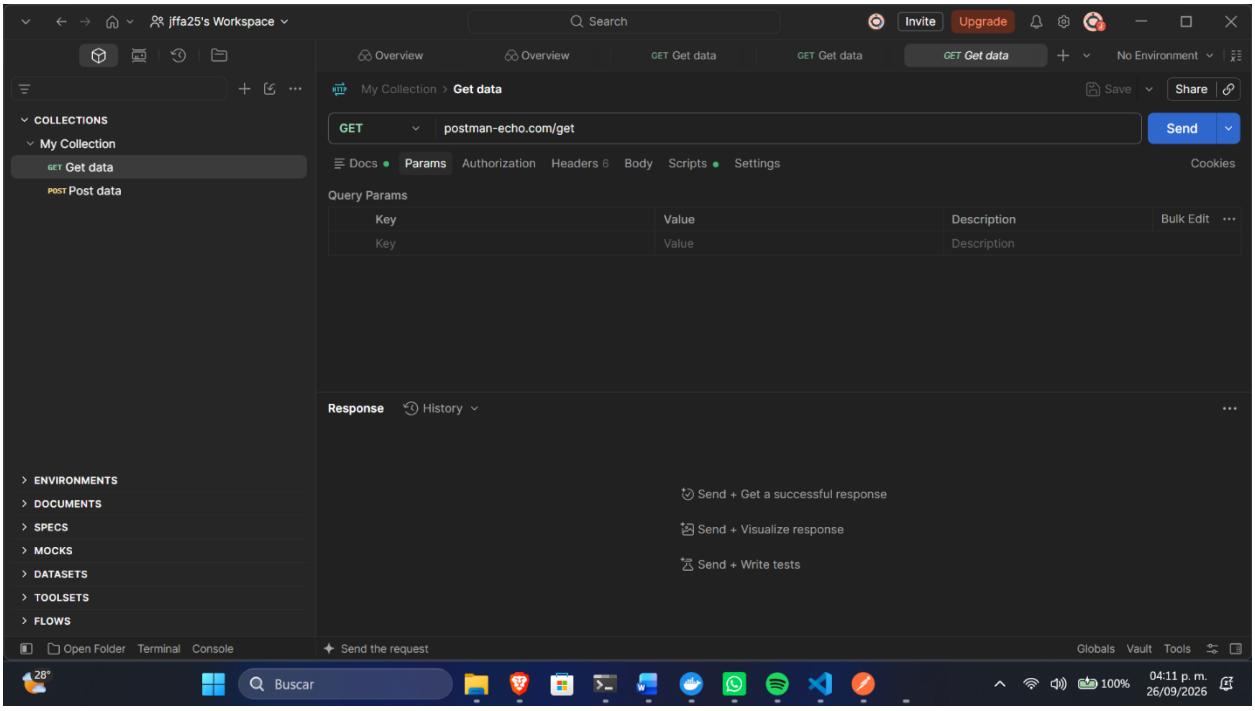


Figura 7. Pantalla principal de Postman tras la instalación

Capítulo 4. Configuración del Entorno Backend

En este capítulo se configuran, mediante contenedores Docker, los servicios de base de datos, el proveedor de identidad y el backend de la aplicación, integrándolos todos en un único archivo docker-compose.yml.

4.1 Configuración de PostgreSQL

```
services:
  postgres:
    image: postgres:16
    container_name: postgres_db
    restart: always
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
      POSTGRES_DB: app_db
    ports:
      - "5432:5432"
    volumes:
      - postgres_data:/var/lib/postgresql/data
```

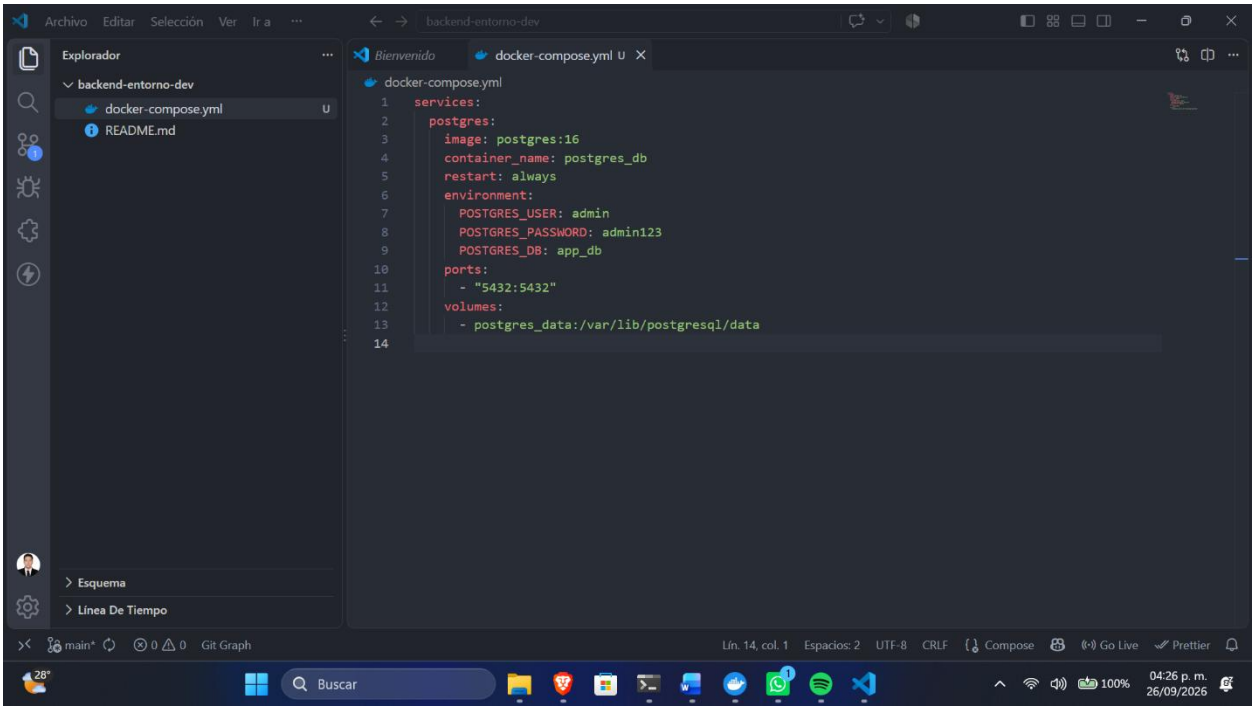


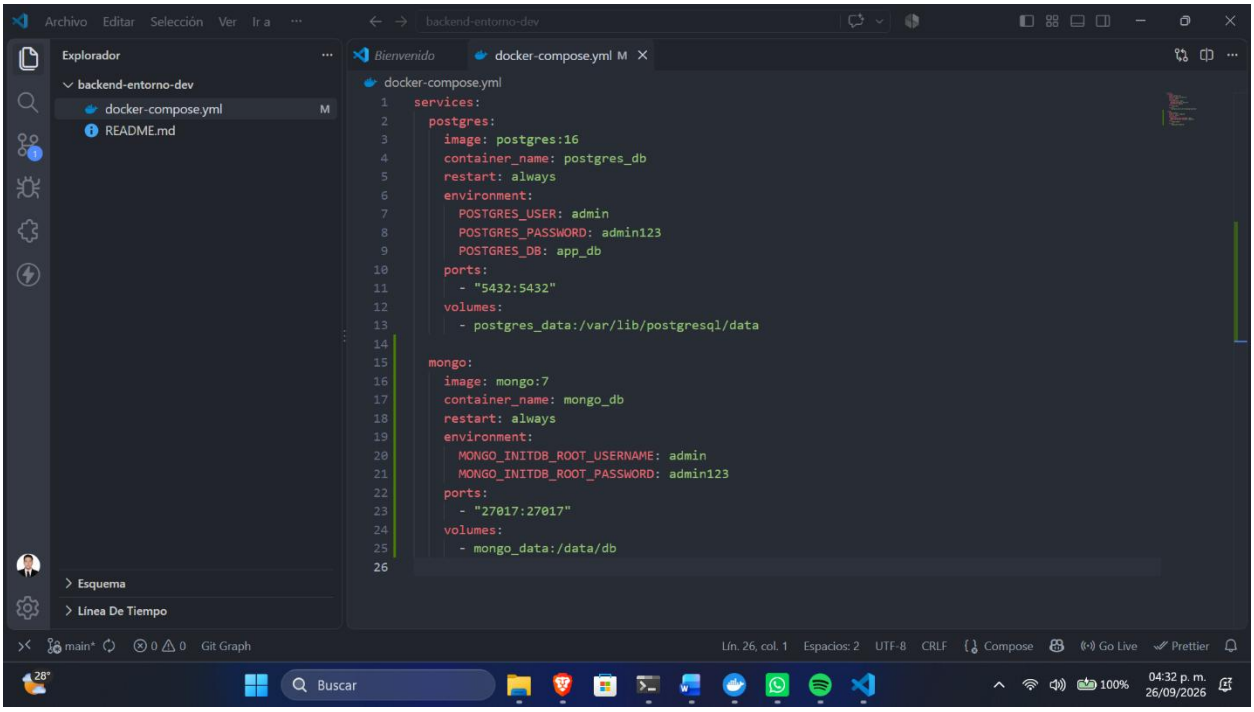
Figura 8. Contenedor de PostgreSQL corriendo, visto con 'docker ps'

4.2 Configuración de MongoDB

Declaramos las configuraciones de mongoDB

```
mongo:
  image: mongo:7
  container_name: mongo_db
  restart: always
  environment:
    MONGO_INITDB_ROOT_USERNAME: admin
    MONGO_INITDB_ROOT_PASSWORD: admin123
  ports:
    - "27017:27017"
  volumes:
    - mongo_data:/data/db
```

Asi se vería en el editor de código de visual studio code



Ejecutamos Docker compse up -d para levantar los contenedores

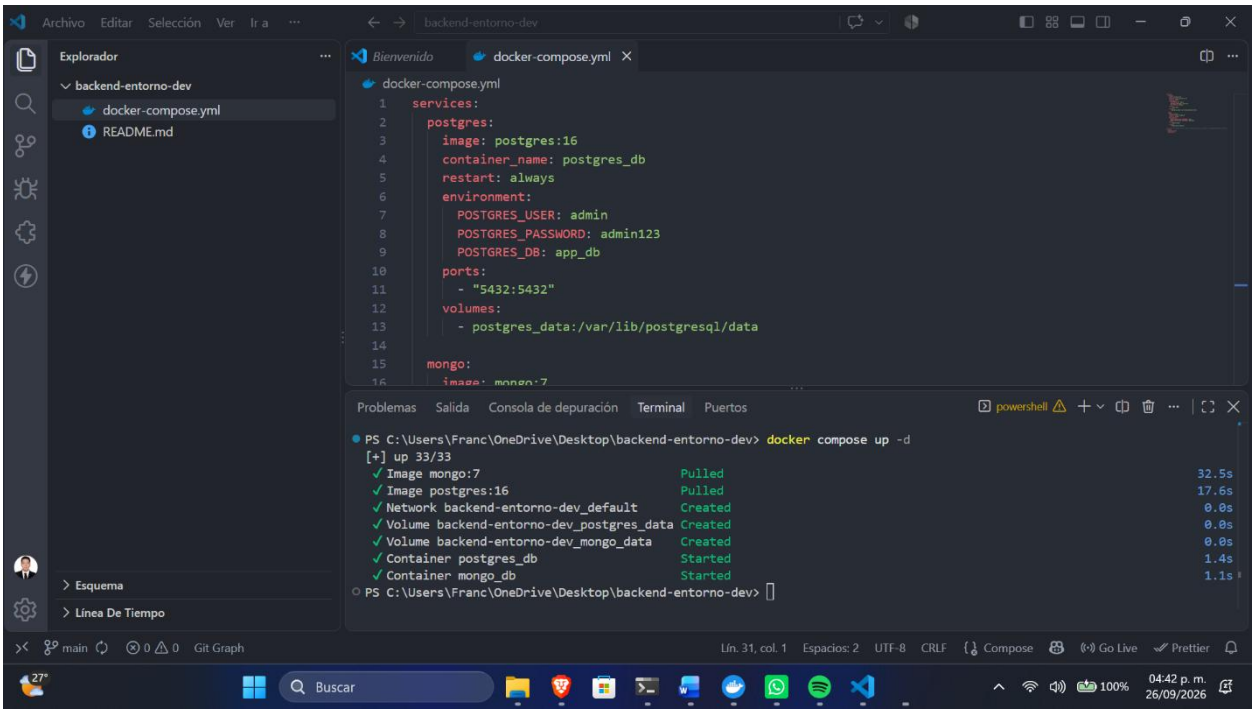
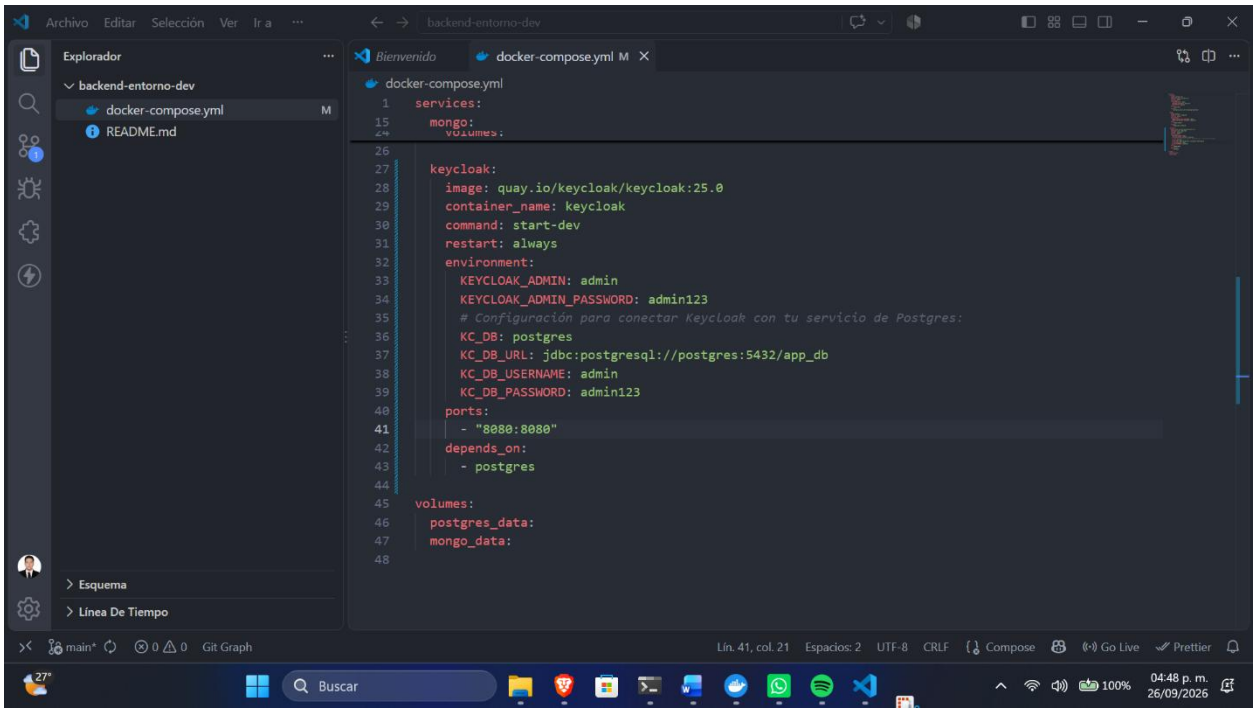


Figura 9. Conexión exitosa a MongoDB mediante MongoDB Compass o mongosh

4.3 Configuración de Keycloak

```
keycloak:
  image: quay.io/keycloak/keycloak:25.0
  container_name: keycloak
  command: start-dev
  environment:
    KEYCLOAK_ADMIN: admin
    KEYCLOAK_ADMIN_PASSWORD: admin123
  ports:
    - "8080:8080"
```



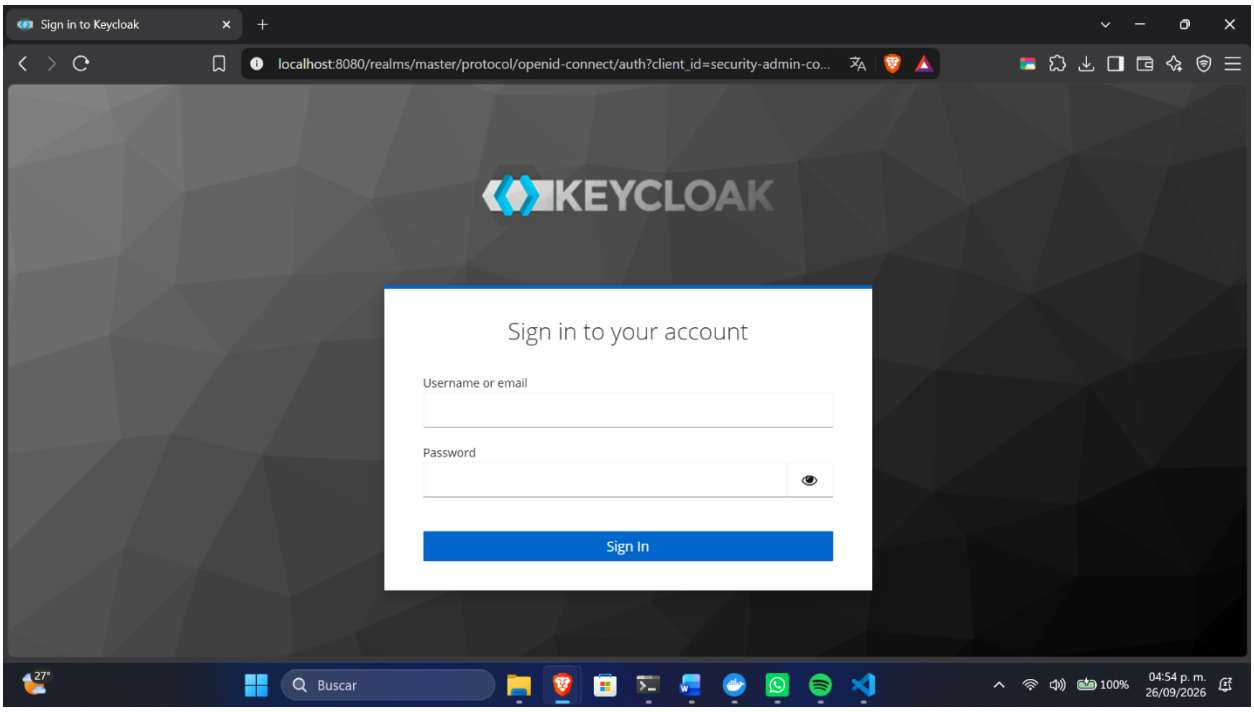


Figura 10. Consola de administración de Keycloak (http://localhost:8080)

Creación de Realm, Cliente y Usuario de prueba

1. Ingresar a la consola de administración con las credenciales configuradas.
2. Crear un nuevo Realm (por ejemplo, 'app-movil').
3. Dentro del Realm, crear un Cliente con Client authentication habilitado y definir las URLs de redirección de la app Flutter.
4. Crear un Rol (por ejemplo, 'usuario' y 'administrador').
5. Crear un Usuario de prueba y asignarle el rol correspondiente.

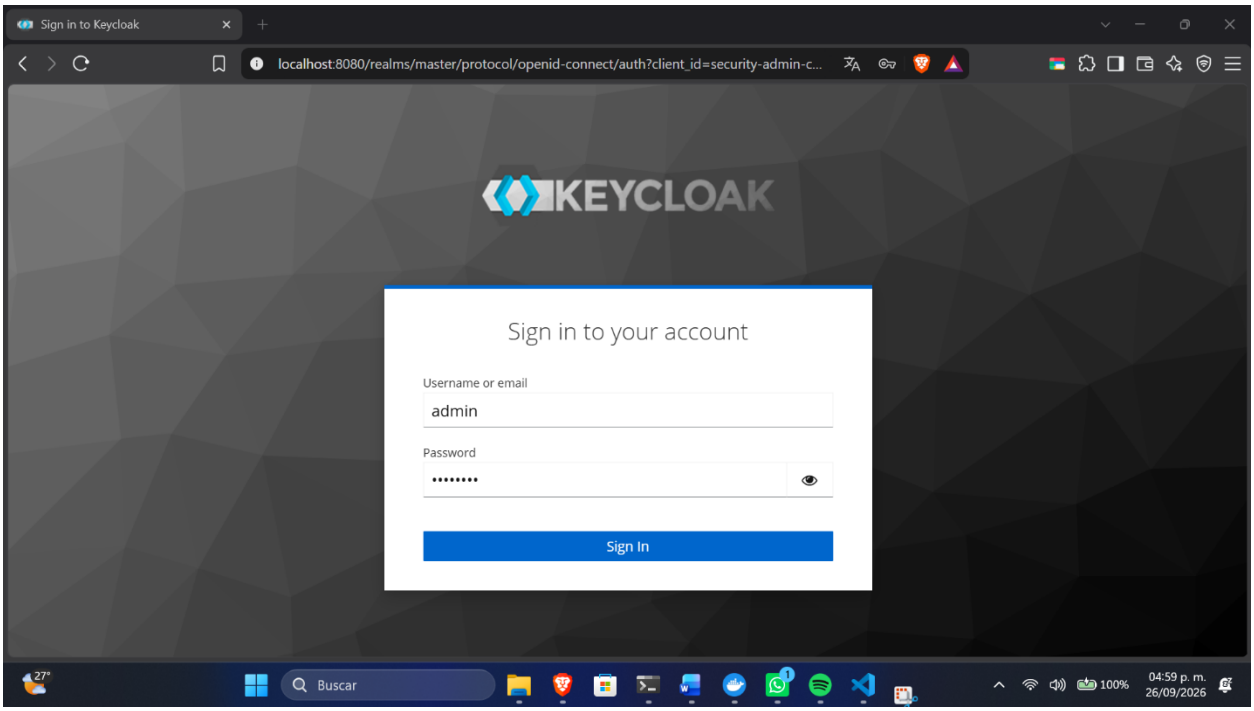


Figura 11. Realm y Cliente creados en Keycloak con la configuración de redirect URIs

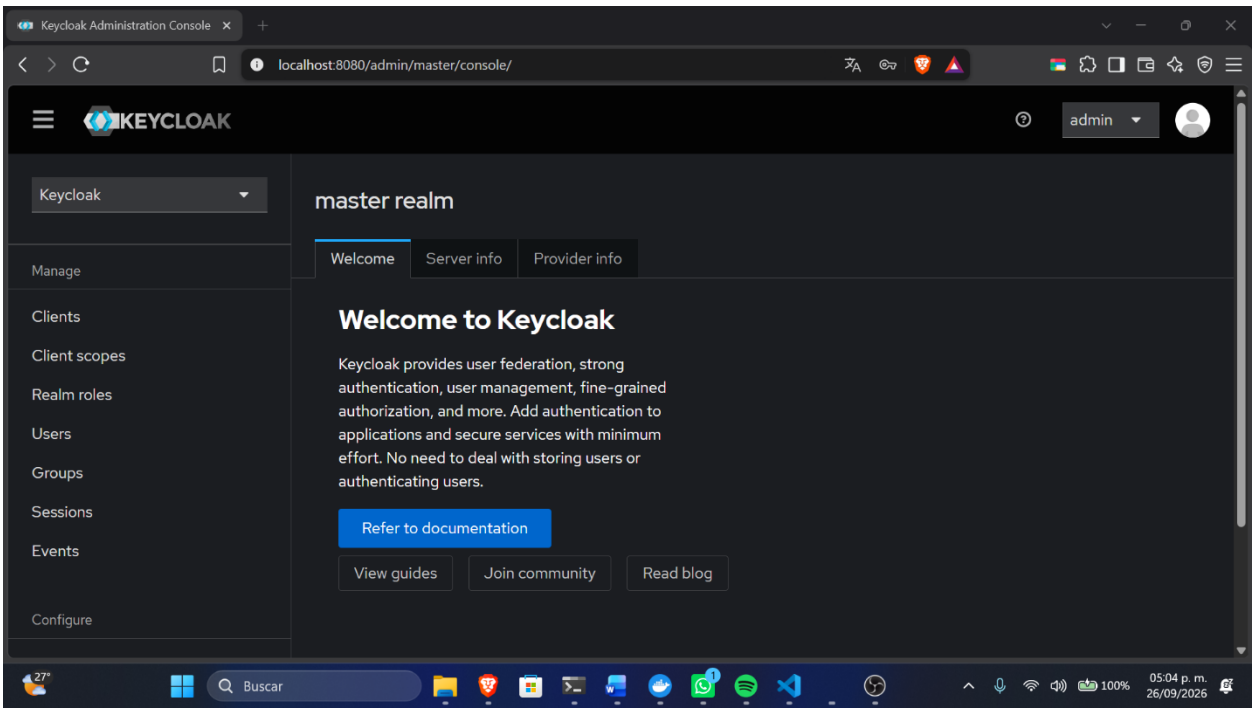
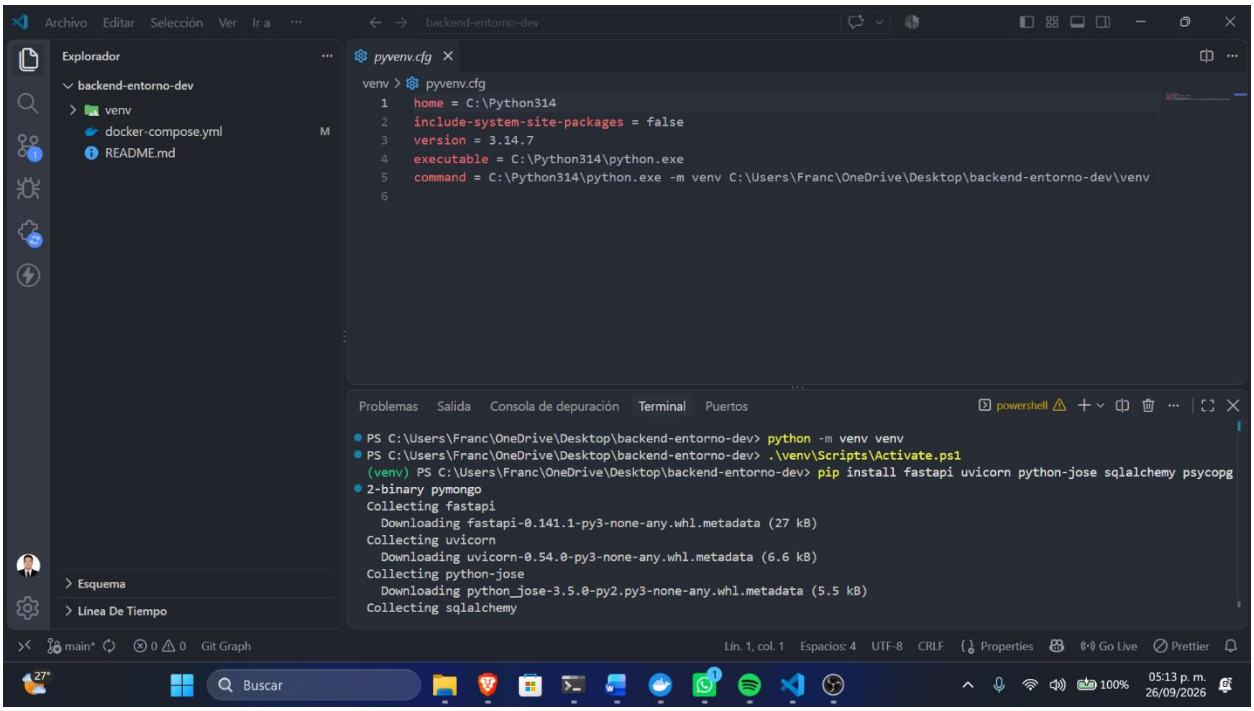


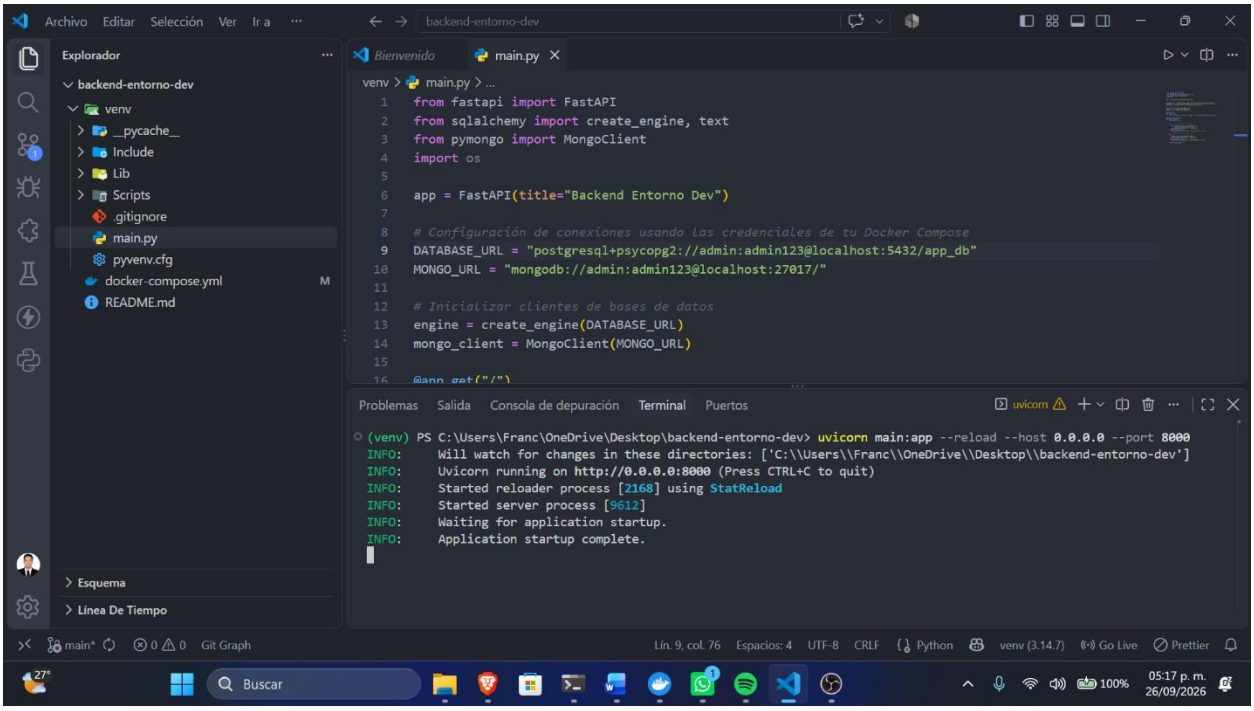
Figura 12. Usuario de prueba creado con su rol asignado

4.4 Configuración de FastAPI

```
python -m venv venv
source venv/bin/activate   # En Windows: venv\Scripts\activate
pip install fastapi uvicorn python-jose sqlalchemy psycopg2-binary pymongo

# Ejecutar el servidor
uvicorn main:app --reload --host 0.0.0.0 --port 8000
```





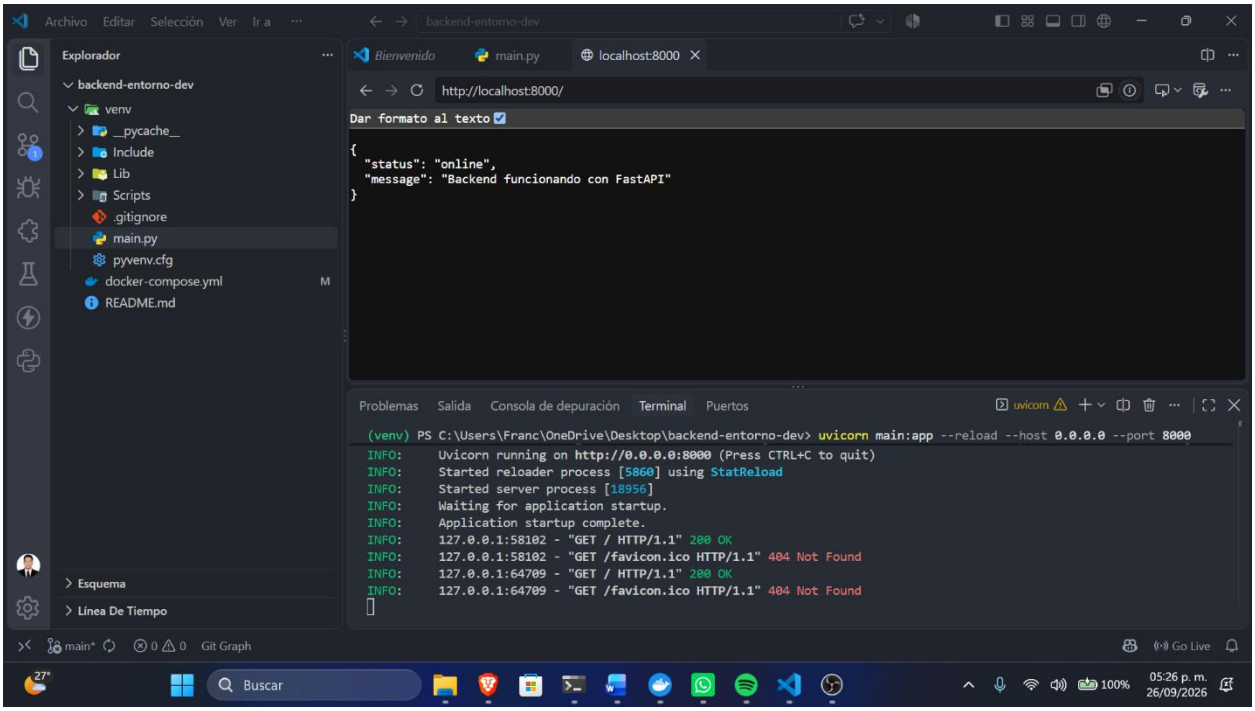


Figura 13. Servidor FastAPI ejecutándose en la terminal (Uvicorn running on http://0.0.0.0:8000)

4.5 Configuración alterna con Node/Express

```
npm init -y
npm install express cors dotenv pg mongoose jsonwebtoken
node index.js
```

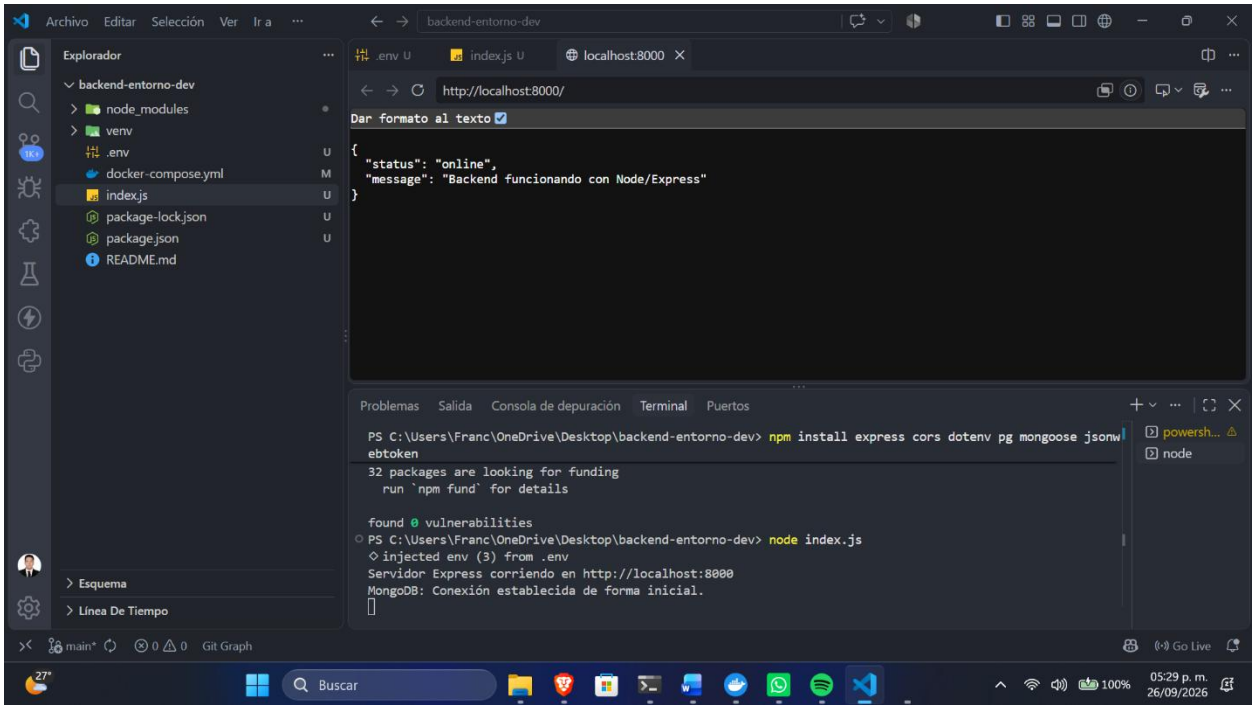


Figura 14. Servidor Express escuchando en el puerto configurado

4.6 Integración de Swagger/OpenAPI

FastAPI genera automáticamente la documentación interactiva en las rutas /docs (Swagger UI) y /redoc, a partir de los tipos y modelos definidos en el código. En Express, esta documentación se agrega manualmente mediante la librería swagger-jsdoc y swagger-ui-express.

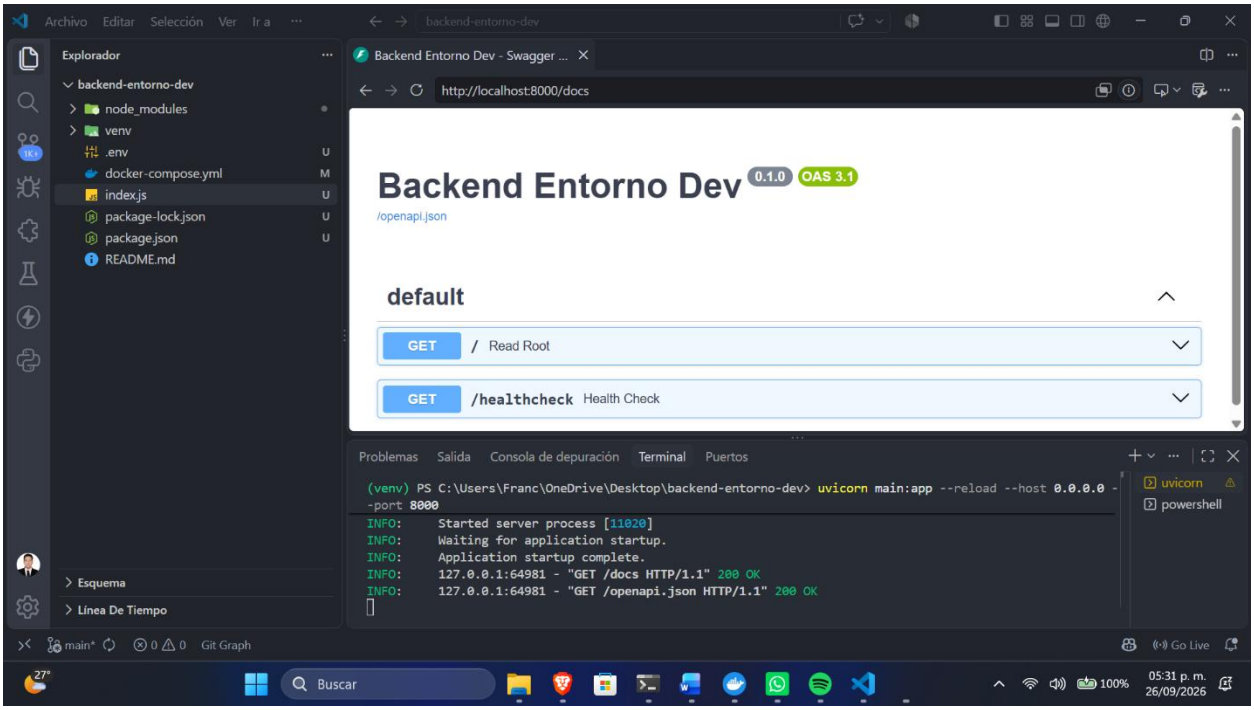


Figura 15. Documentación interactiva de Swagger UI en /docs mostrando los endpoints disponibles

4.7 Orquestación completa con Docker Compose

El archivo final agrupa todos los servicios anteriores, permitiendo levantarlos con un solo comando.

```
docker compose up -d
docker compose ps
docker compose logs -f api
```

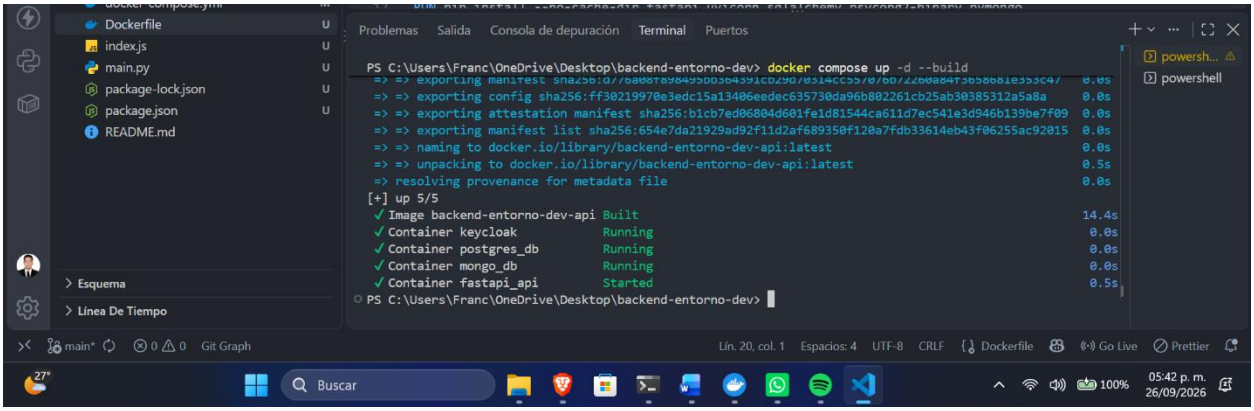


Figura 16. Salida de 'docker compose ps' con todos los servicios en estado 'Up'

Capítulo 5. Configuración del Entorno Frontend (Flutter)

5.1 Creación del proyecto Flutter

```
flutter create app_movil
cd app_movil
flutter run
```



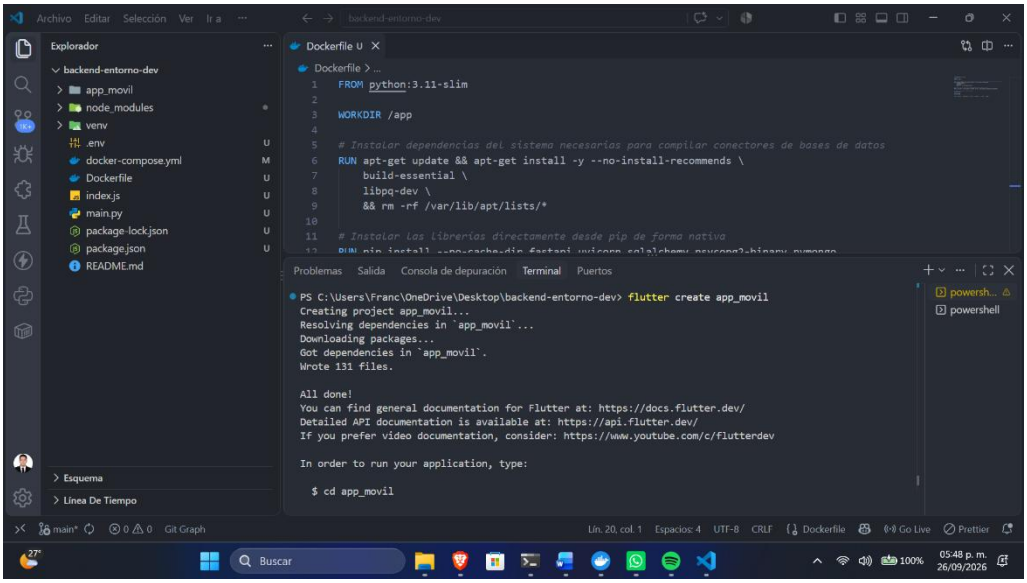


Figura 17. Proyecto Flutter creado y ejecutándose en un emulador/dispositivo

5.2 Estructura de carpetas recomendada

```
lib/
  ├── core/          # constantes, temas, utilidades
  ├── data/          # modelos y servicios de datos (API, auth)
  ├── domain/        # lógica de negocio
  ├── presentation/  # pantallas y widgets
  └── main.dart
```

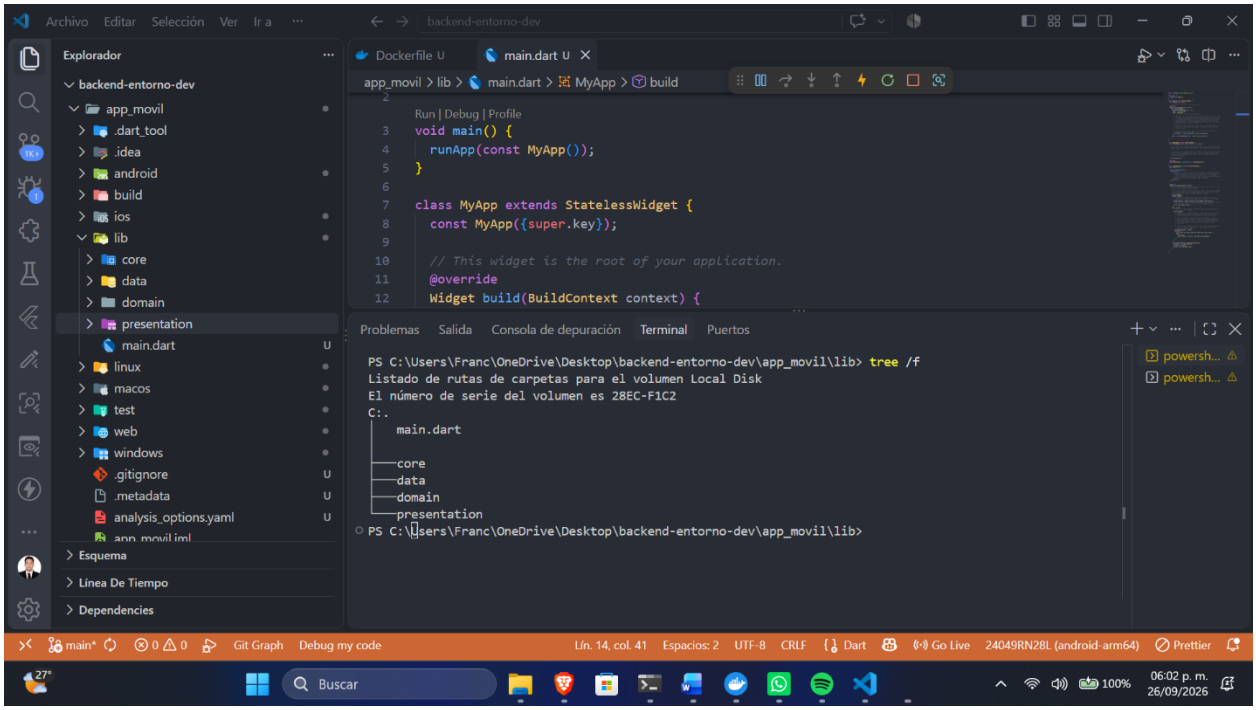
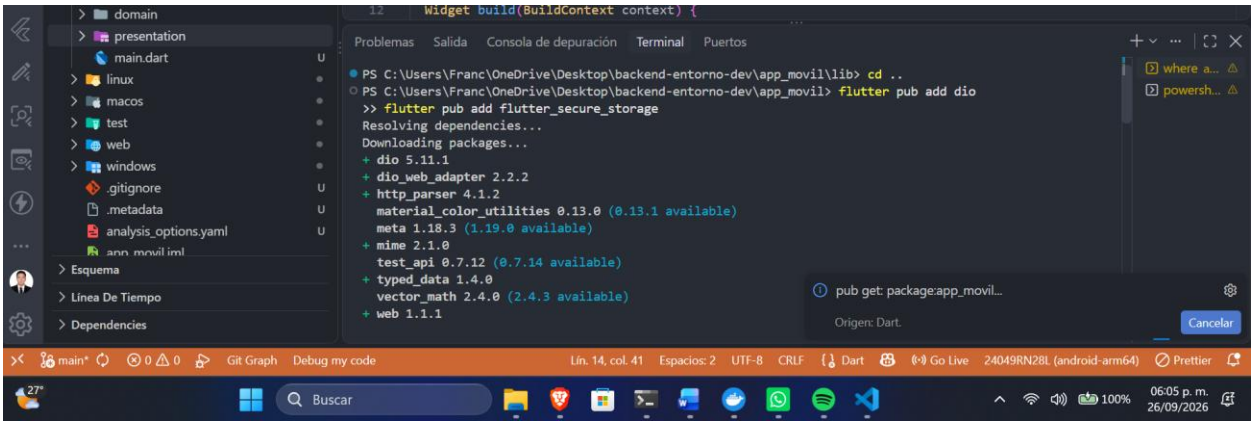


Figura 18. Estructura de carpetas del proyecto vista en el IDE

5.3 Integración de paquetes para consumo de API

```
flutter pub add dio
flutter pub add flutter_secure_storage
```



5.4 Integración de autenticación con Keycloak

```
flutter pub add openid_client
# o alternativamente
flutter pub add flutter_appauth
```

Se configura el cliente OIDC con la URL del Realm de Keycloak, el client_id y las redirect URIs correspondientes, para obtener el token de acceso (access_token) tras el login del usuario.

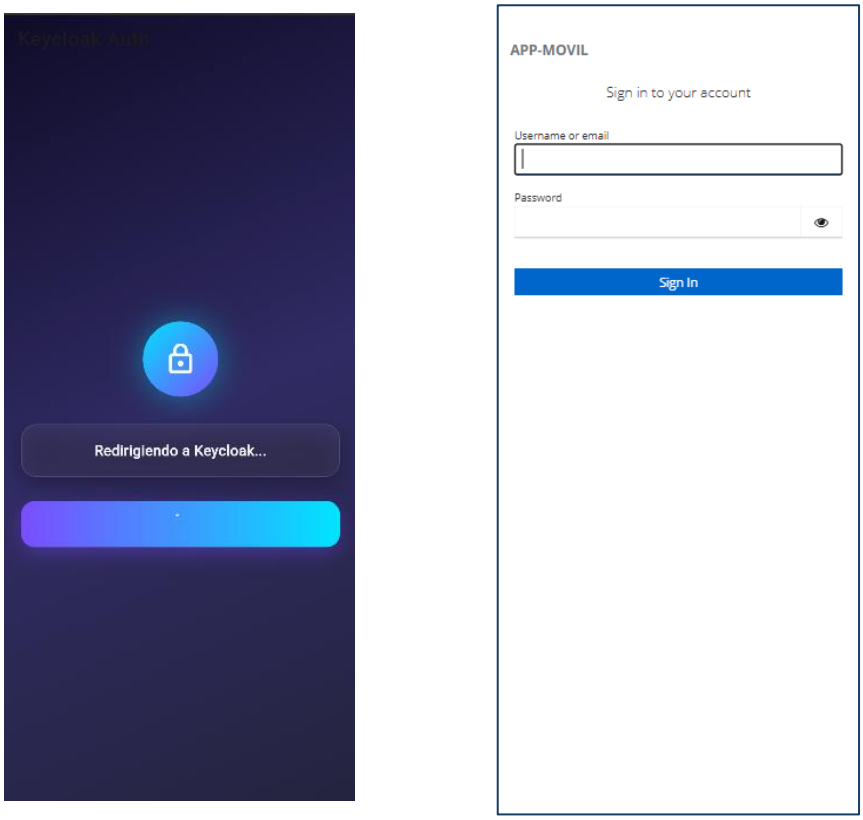


Figura 19. Pantalla de login de Keycloak abierta desde la app Flutter

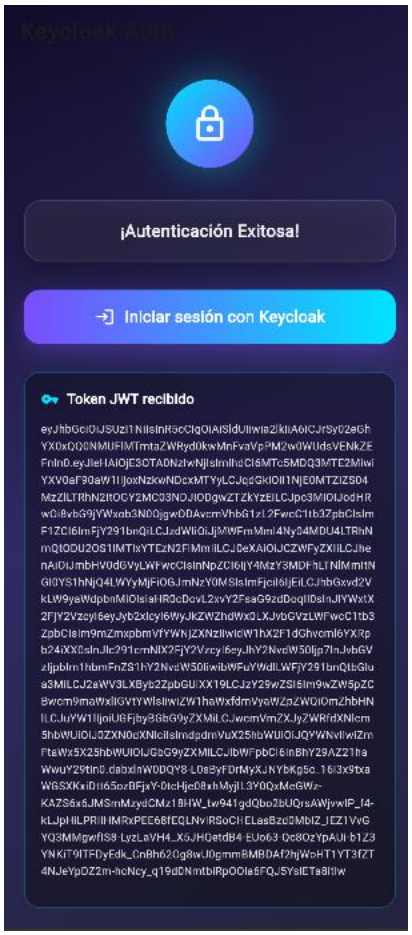
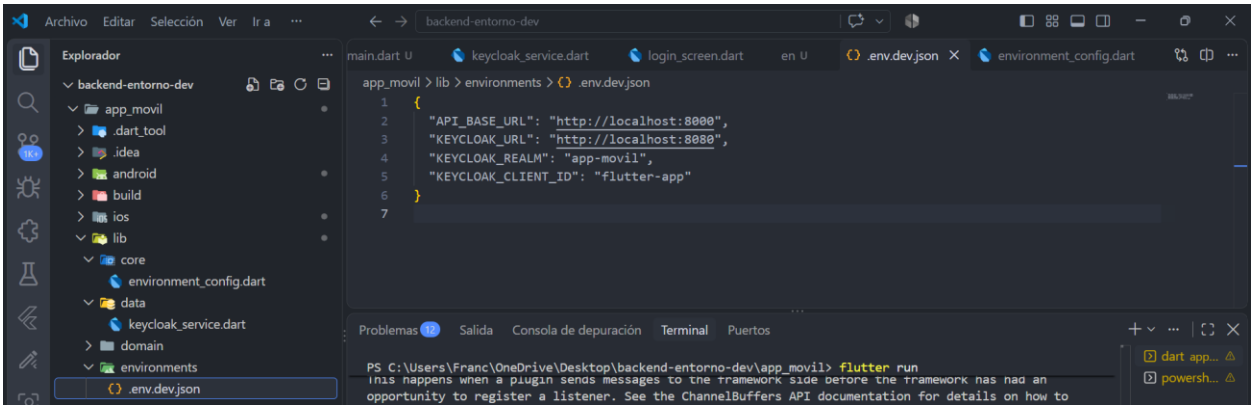


Figura 20. Token JWT recibido correctamente tras la autenticación exitosa

5.5 Configuración de variables de entorno por ambiente

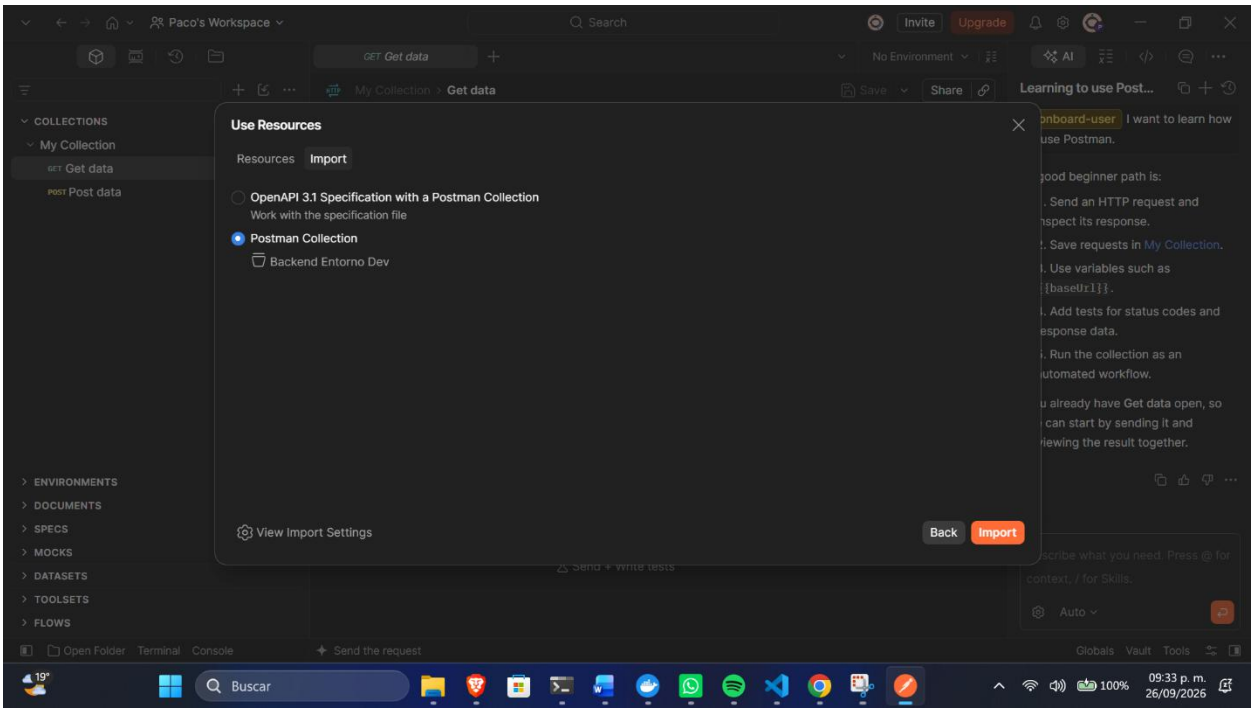
```
// .env.dev
API_BASE_URL=http://localhost:8000
KEYCLOAK_URL=http://localhost:8080
KEYCLOAK_REALM=app-movil
KEYCLOAK_CLIENT_ID=app-movil-client
```



Capítulo 6. Pruebas y Verificación

6.1 Pruebas de endpoints con Postman

6. Crear una nueva colección en Postman para el proyecto.
7. Configurar un entorno (Environment) con variables como base_url y token.
8. Importar la especificación OpenAPI generada por FastAPI/Swagger directamente en Postman.



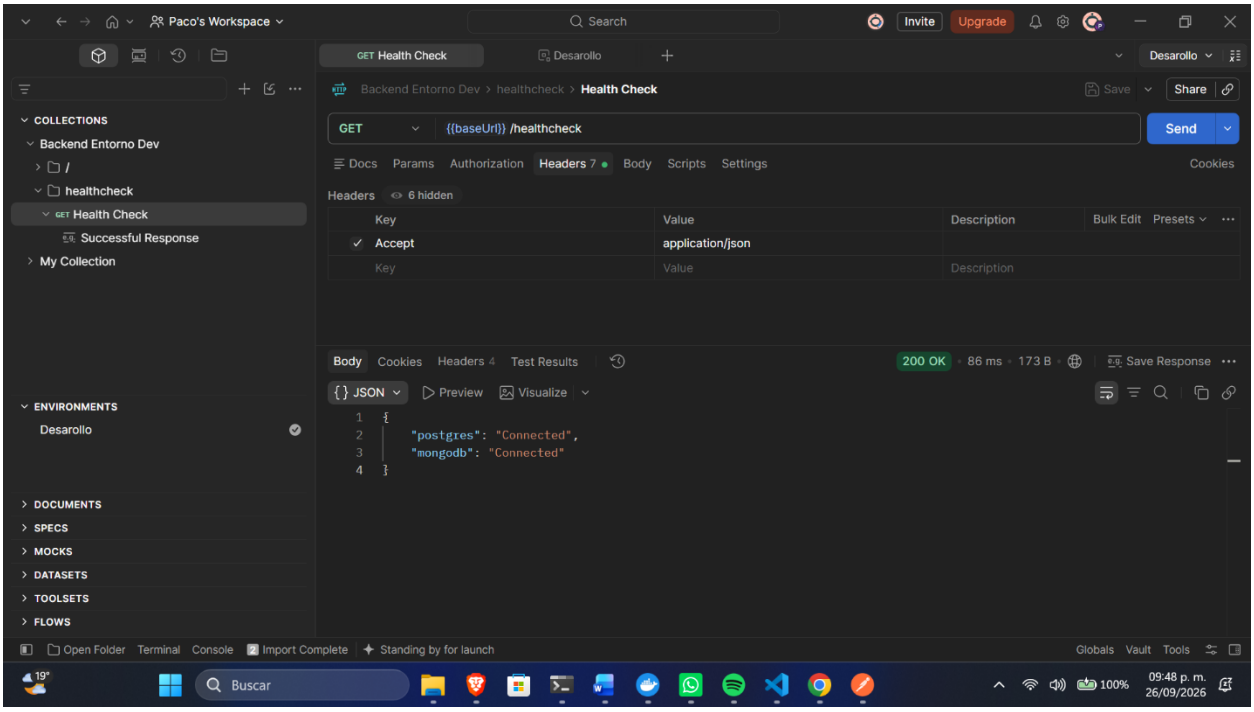


Figura 21. Colección de Postman con las peticiones importadas desde Swagger

6.2 Pruebas de autenticación (obtención y validación de JWT)

Se realiza una petición POST al endpoint de token de Keycloak (/realms/{realm}/protocol/openid-connect/token) utilizando grant_type=password o el flujo configurado, y se valida que el token recibido sea aceptado por los endpoints protegidos del backend.

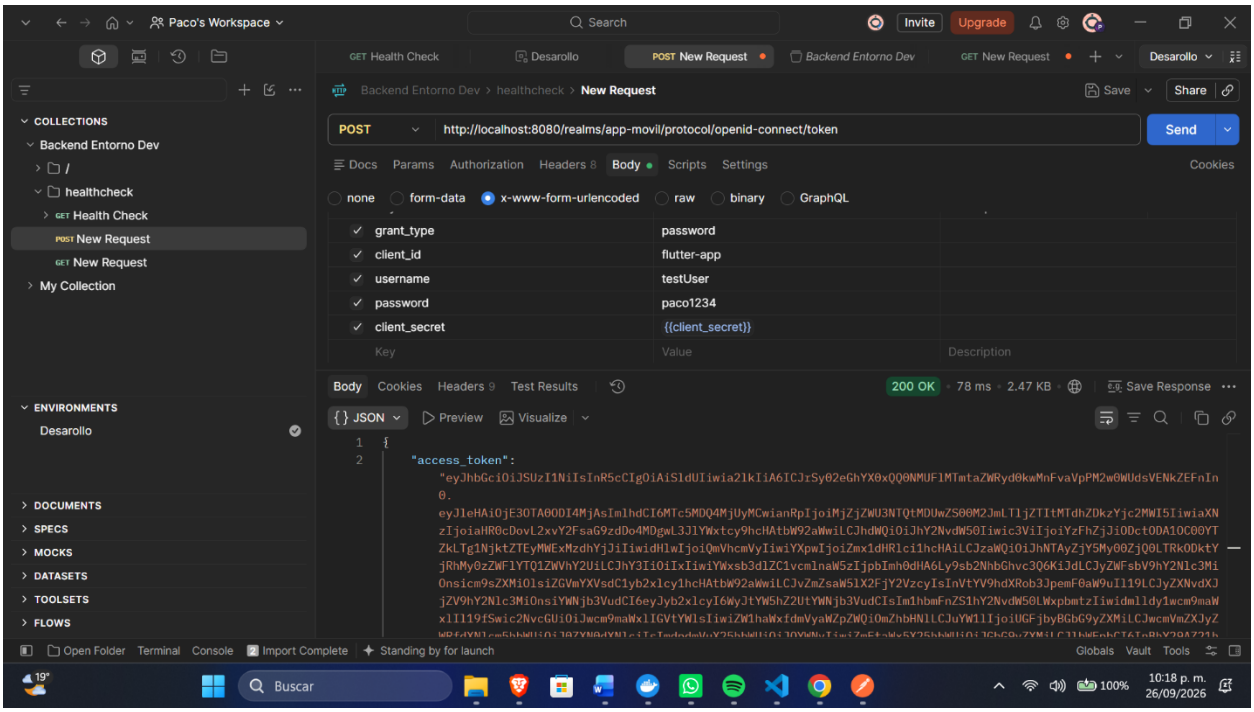


Figura 22. Petición en Postman obteniendo el access_token desde Keycloak

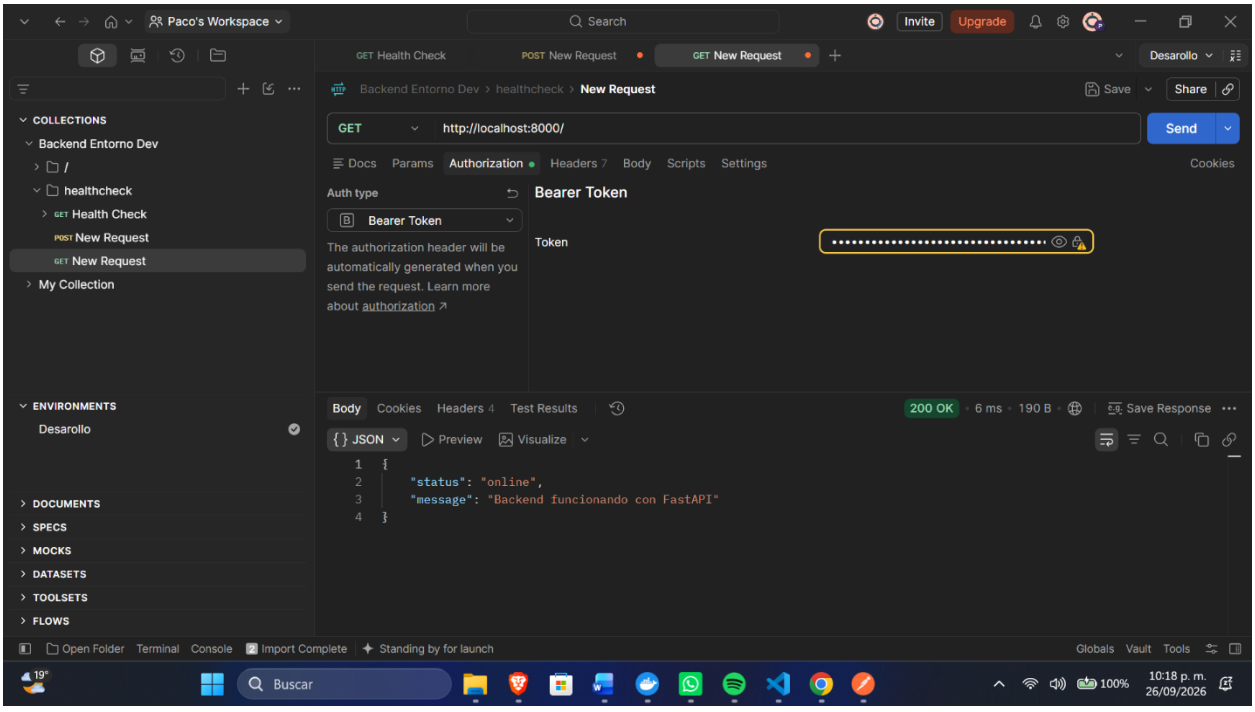


Figura 23. Petición autenticada exitosa (200 OK) usando el token en el header Authorization

6.3 Verificación de la documentación Swagger

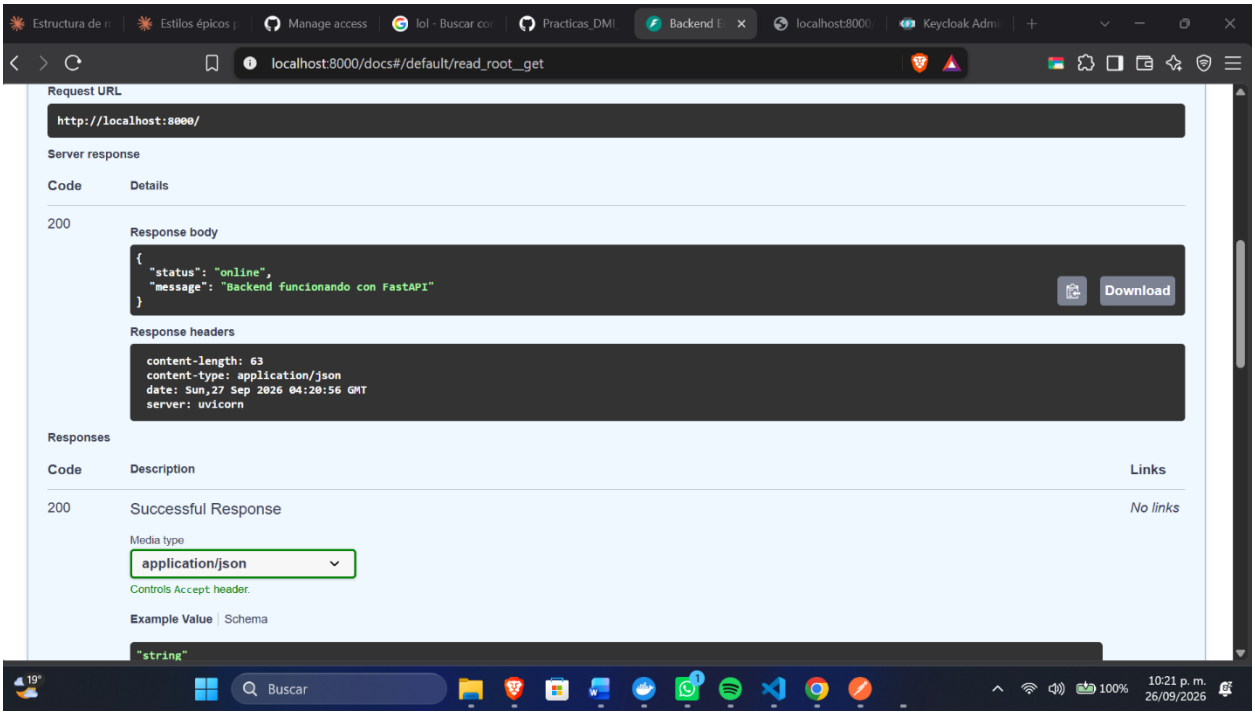


Figura 24. Prueba de un endpoint directamente desde Swagger UI ('Try it out')

6.4 Pruebas de la app Flutter contra el backend

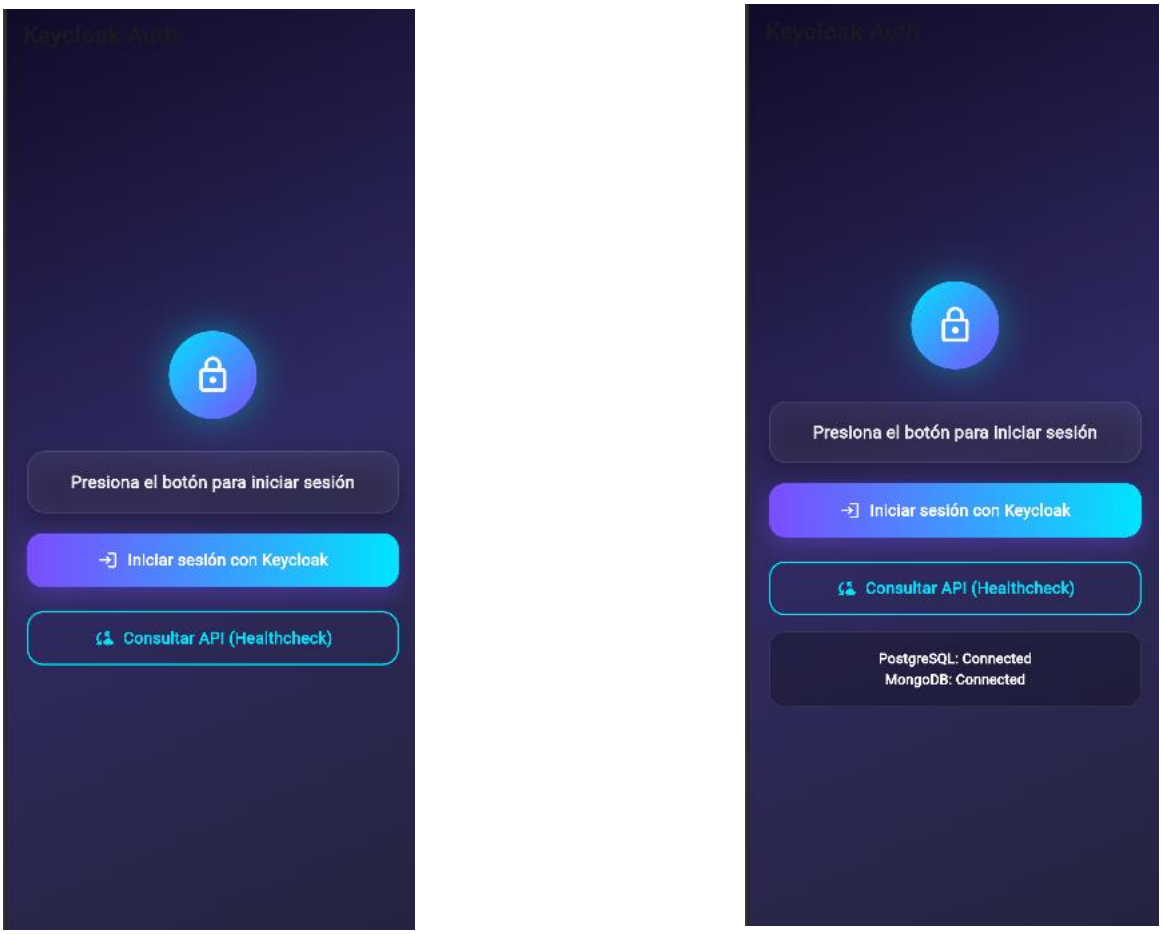


Figura 25. App Flutter mostrando datos obtenidos desde la API en tiempo real

Capítulo 7. Buenas Prácticas

7.1 Manejo de variables sensibles

Nunca se deben subir credenciales al repositorio. Se recomienda un archivo .env (excluido mediante .gitignore) y, en producción, un gestor de secretos.

```
# .gitignore
.env
node_modules/
venv/
*.pyc
build/
```

7.2 Convenciones de commits y ramas

- main / develop: ramas estables.
- feature/nombre-funcionalidad: nuevas funcionalidades.
- fix/nombre-error: corrección de errores.
- Mensajes de commit siguiendo Conventional Commits: feat:, fix:, docs:, chore:.

7.3 Versionado semántico de la API

Se recomienda prefijar las rutas con la versión (por ejemplo, /api/v1/) para permitir cambios futuros sin romper compatibilidad con clientes existentes.

Capítulo 8. Solución de Problemas Comunes

Problema	Causa probable	Solución
Puerto ya en uso (5432, 8080, etc.)	Otro servicio o contenedor ocupando el puerto	Cambiar el mapeo de puertos en docker-compose.yml o detener el proceso con 'docker ps' / 'lsof -i :puerto'
'flutter doctor' con errores	SDK de Android o licencias no aceptadas	Ejecutar 'flutter doctor --android-licenses' y reinstalar componentes faltantes
Fallo de conexión a Keycloak desde la app	Redirect URI mal configurada	Verificar que la URI registrada en el Cliente de Keycloak coincida exactamente con la de la app
Token JWT rechazado por el backend	Emisor (issuer) o audiencia no coinciden	Revisar la configuración del validador de JWT en FastAPI/Express contra la del Realm
Contenedor se reinicia constantemente	Variables de entorno faltantes o incorrectas	Revisar logs con 'docker compose logs -f nombre_servicio'

Anexos

Anexo A. Comandos rápidos de referencia

```
docker compose up -d          # Levantar todos los servicios
docker compose down           # Detener y eliminar contenedores
docker compose logs -f        # Ver logs en tiempo real
docker ps                     # Listar contenedores activos
flutter run                   # Ejecutar la app
flutter pub get               # Instalar dependencias
git status / git add . / git commit -m "mensaje" / git push
```

Anexo B. Ejemplo completo de docker-compose.yml

```
version: '3.9'
services:
  postgres:
    image: postgres:16
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
      POSTGRES_DB: app_db
    ports: ['5432:5432']
  mongo:
    image: mongo:7
    ports: ['27017:27017']
  keycloak:
    image: quay.io/keycloak/keycloak:25.0
    command: start-dev
    ports: ['8080:8080']
  api:
    build: ./backend
    ports: ['8000:8000']
    depends_on: [postgres, mongo, keycloak]
```

Anexo C. Glosario de términos

- API REST: interfaz que expone operaciones sobre recursos mediante HTTP.
- JWT: JSON Web Token, formato compacto para transmitir información de identidad de forma segura.
- Contenedor: unidad ejecutable que empaqueta una aplicación y sus dependencias.
- Realm (Keycloak): espacio aislado de configuración de seguridad e identidad.
- Endpoint: URL específica de una API que responde a una operación concreta.

Anexo D. Referencias

Documentación oficial de Flutter (flutter.dev), Docker (docs.docker.com), Keycloak (www.keycloak.org), FastAPI (fastapi.tiangolo.com), PostgreSQL (www.postgresql.org), MongoDB (www.mongodb.com), Swagger/OpenAPI (swagger.io) y Postman (www.postman.com).